

Residual Multi-Tree Genetic Programming for Joint Adaptation of State Transition and Pheromone Update Rules in Ant Colony Optimization

First Author, Second Author, and Third Author

Abstract—Ant colony optimization depends on a state transition rule and a pheromone update rule. Both rules are usually designed by experts. Genetic programming can automate this design process. However, direct replacement of a complete rule can remove useful structure from the original algorithm. This study proposes residual multi-tree genetic programming for ant colony optimization, called RMTGP-ACO. It uses two strongly typed trees. The first tree applies a bounded correction to transition desirability. The second tree redistributes pheromone deposits under a fixed event budget. Zero output from both trees exactly recovers the original ACO. We implement the method for Ant System, Ant Colony System, and MAX-MIN Ant System. The first study trains on uniform TSP100 without local search. Full-F1 reduces the mean reference gap in all 12 AS/ACS/MMAS and TSP50/100/500/1000 comparisons. It also outperforms a retrained prior-study representation in all six matched TSP100/TSP500 comparisons. Controlled ablations show that residual integration is not universally better than replacement. They also show no stable universal advantage for two trees, the Full terminal set, or the F1 function set. We then study why 2-opt weakens the GP learning signal. The final-gap variance after 2-opt is only 1.08–3.47% of its pre-2-opt value in the TSP100 audit. We introduce LS-aware terminals, multi-fidelity fitness, and a three-stage TSP500 training procedure. On uniform, clustered, and Gaussian TSP500, the AS model reduces the ACO+2-opt gap by 55.8%, 32.1%, and 50.0%, respectively. It also outperforms ACO+3-opt on the uniform and Gaussian sets, but not on the clustered set. ACS and MMAS do not show the same benefit. The fused CUDA evaluator gives a 14.626-fold single-GPU speedup over a 16-thread CPU reference on the registered workload. All evolutionary comparisons use three GP runs and are treated as pilot evidence.

Index Terms—Ant colony optimization, genetic programming, hyper-heuristic, multi-tree genetic programming, residual learning, traveling salesman problem.

I. INTRODUCTION

ANT colony optimization (ACO) constructs solutions from a pheromone model and problem-specific heuristic information [1], [2]. Its behavior depends on two central components. The state transition rule selects the next solution component. The pheromone update rule changes the sampling model after each iteration. These rules are effective, but they are usually fixed by human design.

Genetic programming (GP) can evolve executable rules [3], [4]. It can therefore reduce the need for manual formula design. Previous GP-ACO work evolved complete state transition rules [5]. That study showed that evolved rules can transfer across traveling salesman problem (TSP) sizes

and distributions. It also showed that global terminals can be useful. However, a complete replacement changes the full transition model. It does not preserve the original balance between pheromone and heuristic information. It can also produce unstable score scales.

Pheromone adaptation creates a second design problem. Earlier work used strongly typed GP to evolve pheromone strategies [6]. This direction is important, but joint transition and pheromone adaptation is difficult. The two rules act at different points in ACO. Their tensors also have different meanings and shapes. A shared untyped tree would allow invalid combinations. Two unrestricted trees would also receive more representational capacity than one tree.

We propose residual multi-tree genetic programming for ACO, or RMTGP-ACO. The method uses one transition tree and one pheromone tree. The trees have separate strong types. The transition tree multiplies the original desirability by a bounded residual factor. The pheromone tree redistributes a fixed deposit budget over the edges that the original ACO would reinforce. The method retains the main semantics of Ant System (AS), Ant Colony System (ACS), and MAX-MIN Ant System (MMAS) [1], [7], [8].

This study makes six technical contributions.

- It defines a bounded residual interface for ACO state transition.
- It defines an event-wise pheromone residual that preserves support and total deposit.
- It introduces a strongly typed two-tree representation with a shared node budget.
- It provides a fused GPU evaluator for the nested GP-ACO computation.
- It diagnoses the loss of final-gap learning signal after 2-opt and introduces LS-aware edge and search-context terminals.
- It provides a staged TSP500 training procedure and compares RMTGP-ACO+2-opt with ACO+2-opt and ACO+3-opt on three distributions.

Each evolutionary condition uses three independent GP runs. The paper reports complete main, ablation, capacity, training-budget, learning-signal, and local-search experiments. The small GP-run count supports directional and failure-mode analysis. It does not support broad confirmatory causal claims.

II. BACKGROUND AND PROBLEM DEFINITION

A. Euclidean Traveling Salesman Problem

Let $V = \{1, \dots, n\}$ be a set of cities. Each city i has a coordinate $c_i \in \mathbb{R}^2$. The distance between cities i and j is

$$d_{ij} = \|c_i - c_j\|_2. \quad (1)$$

A tour is a permutation $\pi = (\pi_1, \dots, \pi_n)$. Its length is

$$L(\pi) = \sum_{q=1}^{n-1} d_{\pi_q, \pi_{q+1}} + d_{\pi_n, \pi_1}. \quad (2)$$

The objective is to minimize $L(\pi)$.

Each dataset row contains a reference tour. We recompute its length in float64. The available files do not provide a global optimality certificate for every tour. We therefore use the term *reference length*, denoted by L_i^{ref} . The primary quality measure is

$$g_i(\theta) = 100 \frac{L_i(\theta) - L_i^{\text{ref}}}{L_i^{\text{ref}}}. \quad (3)$$

We call g_i the reference gap.

B. Baseline ACO State Transition

Let τ_{ij} be the pheromone value of edge (i, j) . Let

$$\eta_{ij} = \frac{1}{d_{ij} + \epsilon_d} \quad (4)$$

be its heuristic value. At construction step q , ant a is at city i . Its feasible set is $F_{a,q}$. The baseline desirability of city $j \in F_{a,q}$ is

$$s_{aij}^0 = \tau_{ij}^\alpha \eta_{ij}^\beta. \quad (5)$$

AS and MMAS sample from the normalized score. ACS uses the pseudorandom proportional rule. It selects the maximum score with probability q_0 . Otherwise, it samples from the normalized score. All variants use a candidate list. If that list has no feasible city, the implementation uses the full feasible set.

C. Baseline Pheromone Update

AS evaporates all pheromone and reinforces the tours of all ants. For ant a , the baseline deposit on a tour edge is $1/L_a$. ACS applies its local update during construction. It then applies a global update to the global-best tour. MMAS reinforces one scheduled tour. The schedule uses iteration-best tours and restart-best tours. MMAS also applies dynamic pheromone bounds and restart logic.

Our implementation follows the algorithmic semantics of ACOTSP 1.03 [9]. All three variants use 32 ants and 500 iterations in this study. Local search is disabled.

D. Previous GP-ACO Baseline

The previous study evolved one complete state transition expression [5]. We denote a matched implementation of that approach by Legacy-GP. Its terminals are raw pheromone, raw distance, mean pheromone, mean distance, problem size, and feasible-set size. It uses arithmetic primitives and a protected division that returns one when the denominator is close to zero.

The present study retrains Legacy-GP under the current data, ACO budget, and random schedules. Published numerical results are not copied into the current paired tables. This rule prevents a comparison between incompatible budgets.

III. RESIDUAL MULTI-TREE GP-ACO

A. Overview

An RMTGP-ACO individual contains two programs:

$$\theta = (T_{\text{tr}}, T_{\text{ph}}). \quad (6)$$

The transition program acts during solution construction. The pheromone program acts after the colony tours are available. Fig. 1 shows the data flow.

B. Strongly Typed Representation

The implementation uses DEAP strongly typed GP [10], [11]. It defines two distinct Python marker classes: `TrField` and `PhField`. The two `PrimitiveSetTyped` root signatures are `TRANSITION: () -> TrField` and `PHEROMONE: () -> PhField`. Every subtree inside one tree has the same field type as its root. DEAP therefore rejects subtree exchange between the two semantic roles.

DEAP checks only the `TrField/PhField` marker during tree construction, crossover, and mutation. The shapes $[B, M, K]$ and $[B, R, n]$ are runtime contracts maintained by the evaluator. They are not additional DEAP static types. This separation allows B, K, R, n to change with the batch, ACO variant, and TSP size while preserving semantic type safety.

Each ACO task in B binds one program to one TSP instance. When one individual is evaluated, B is the instance batch. During fused population evaluation, B is the program-instance task batch. Usually, $K = \min(20, n - 1)$. Candidate-list fallback uses $K = n$. AS has $R = M = 32$. ACS has $R = 1$ and uses the global-best source tour. MMAS has $R = 1$ and retains its original reinforcement schedule.

The empty argument list in each root signature does not mean that a tree has no information. Each named terminal is a zero-arity typed leaf that is bound to the current ACO context at runtime. The evaluator constructs only terminals that are referenced by the program. The PyTorch backend can materialize the full fields. The fused CUDA and Numba backends compute the same logical values in flattened or compact form. The reported shapes are therefore logical tensor shapes. They do not require every terminal to be materialized at the same time.

If a fused call evaluates P deduplicated programs on I instances, then $B = PI$. For TSP100 with $M = 32$ and $K = 20$, the transition field is $[PI, 32, 20]$. The AS pheromone

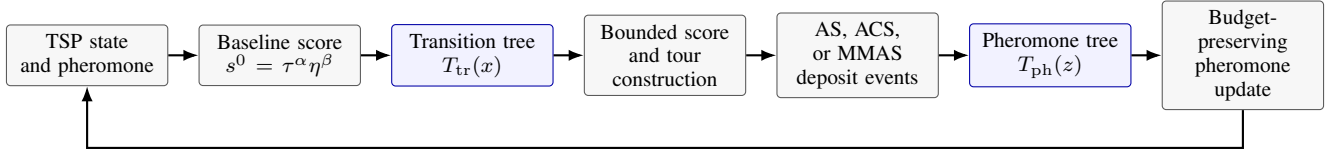


Fig. 1. RMTGP-ACO data flow. The blue boxes are evolved programs. The other boxes retain the semantics of the selected ACO variant.

TABLE I
INPUT, OUTPUT, AND EXECUTION LOCATION OF THE TWO TYPED GP TREES

Tree	Strong type and shape	Dimension meaning	Input–output contract	Execution point
Transition tree	TrField, $[B, M, K]$	B : ACO task batch; M : ants; K : current candidate width	Maps a dynamic selection context to one candidate-level output field	Every tour-construction step
Pheromone tree	PhField, $[B, R, n]$	R : reinforcement events; n : edges in one closed tour	Maps a dynamic reinforcement context to one event–edge output field	Reinforcement stage of every ACO iteration

TABLE II
RUNTIME CONTEXT AVAILABLE TO EACH TREE IN ONE CALL

Context	Raw information and shape	Meaning of one output element
Transition	Distance d , heuristic η , and pheromone τ : $[B, n, n]$; current city c : $[B, M]$; candidate city C and feasible mask F : $[B, M, K]$; baseline probability p^0 : $[B, M, K]$; construction step q , ACO iteration t , budget T , and stagnation counter $s[B]$	X_{bak}^{tr} is one terminal value for task b , ant a , and candidate slot k . C_{bak} maps that slot to a city.
Pheromone reinforcement	d, η, τ , and full nearest-neighbor rank: $[B, n, n]$; source tour π^{src} : $[B, R, n + 1]$; expanded endpoints u, v : $[B, R, n]$; source length: $[B, R]$; colony tours: $[B, M, n + 1]$; colony lengths: $[B, M]$; t, T , and $s[B]$	X_{bre}^{ph} is one terminal value for edge position e of reinforcement event r in task b .

field is $[PI, 32, 100]$. The ACS and MMAS pheromone fields are $[PI, 1, 100]$.

The two trees share one fitness. They also share a total size limit:

$$N_{tr} + N_{ph} \leq 31. \quad (7)$$

An inactive zero tree has zero effective nodes. Each tree also has a maximum depth of five. Equation (7) is the active constraint in the main study. The two trees therefore share 31 effective nodes in total. The LS-aware TSP500 extension uses the same shared-budget rule with $N_{tr} + N_{ph} \leq 62$ and a maximum depth of seven. The larger limit is assigned to the pair, not to each tree. The limit is not 31 nodes per tree. This rule prevents a two-tree individual from receiving twice the capacity of a one-tree individual.

C. Transition Residual

The transition tree receives the terminal field x_{aij} . Its raw output is

$$u_{aij} = T_{tr}(x_{aij}). \quad (8)$$

The output changes the baseline score in (5):

$$r_{aij} = \tanh(u_{aij}), \quad (9)$$

$$m_{aij} = 1 + \gamma_{tr} r_{aij}, \quad (10)$$

$$\tilde{s}_{aij} = s_{aij}^0 m_{aij}, \quad (11)$$

where $\gamma_{tr} = 1/3$. The final probability is

$$p_{aij} = \frac{\tilde{s}_{aij}}{\sum_{\ell \in F_{a,q}} \tilde{s}_{a\ell}}. \quad (12)$$

This interface has four useful properties. First, a zero tree gives $m_{aij} = 1$. It exactly recovers the baseline ACO. Second, $m_{aij} \in [2/3, 4/3]$. The score remains positive. Third, infeasible edges remain infeasible. Fourth, for two feasible cities j and k ,

$$\frac{m_{aij}}{m_{aik}} \in [1/2, 2]. \quad (13)$$

The tree can change a local preference, but it cannot create an unbounded score ratio.

D. Pheromone Residual

A deposit event is

$$r = (\pi_r, L_r, w_r). \quad (14)$$

π_r is a source tour. L_r is its length. w_r is the baseline weight. For each edge e in π_r , the baseline deposit is

$$D_{r,e}^0 = \frac{w_r}{L_r}. \quad (15)$$

The total event budget is

$$B_r = \sum_{e \in E(\pi_r)} D_{r,e}^0 = \frac{nw_r}{L_r}. \quad (16)$$

The pheromone tree produces

$$v_{r,e} = T_{ph}(z_{r,e}), \quad (17)$$

$$h_{r,e} = 1 + \gamma_{ph} \tanh(v_{r,e}), \quad (18)$$

where $\gamma_{\text{ph}} = 1/3$. The final deposit is

$$\tilde{D}_{r,e} = B_r \frac{D_{r,e}^0 h_{r,e}}{\sum_{f \in E(\pi_r)} D_{r,f}^0 h_{r,f}}. \quad (19)$$

The zero tree recovers the baseline deposit. The reinforced edge set is unchanged. The event budget is also conserved:

$$\sum_{e \in E(\pi_r)} \tilde{D}_{r,e} = B_r. \quad (20)$$

AS creates one event for each ant. ACS creates one global-best event. MMAS creates one event from its original update schedule. The residual only redistributes each event. It does not modify ACS local updates, MMAS bounds, or MMAS restart decisions.

E. Residual Strength

Both interfaces use the same residual strength, $\gamma = 1/3$. In general, the multiplier is

$$m(y) = 1 + \gamma \tanh(y), \quad 1 - \gamma \leq m(y) \leq 1 + \gamma. \quad (21)$$

Any $\gamma < 1$ preserves a positive multiplier. For $\gamma = 1/3$, one multiplier is bounded by $2/3$ and $4/3$. The ratio between the multipliers of two candidates is at most two. After probability or deposit-share normalization, an original share can change by at most a factor from $1/2$ to 2 . This range allows GP to change a relative preference while retaining the baseline rule as the dominant component.

The value $1/3$ was fixed before the test results were inspected. It was not selected on a test set. A value of 0.1 would allow only a weak correction. A value of 0.5 would allow a threefold relative correction between two candidates. We use the same value for both trees to remove one potential confounder. The current ablation does not treat γ as a factor. A future sensitivity study can compare 0.1 , $1/3$, and 0.5 .

F. Terminal Design Principles

The basic terminals in the previous GP-ACO study were pheromone concentration and distance [5]. Its extended configuration also included mean candidate pheromone, mean candidate distance, the number of cities, and the number of unvisited cities. Those terminals are suitable when GP must generate a complete transition score. We do not copy that set unchanged because the present trees learn residuals. The baseline score $s^0 = \tau^\alpha \eta^\beta$ already preserves the main absolute relation between pheromone and distance. The residual tree does not need to reconstruct that relation.

Table III maps information roles. It does not claim algebraic equivalence between the two terminal sets. We remove absolute means and raw city counts because they shift with the pheromone stage and problem size. The baseline ACO state still retains their effects. Our hypothesis is that relative inputs transfer more easily across sizes. Legacy-GP compares the two overall representations, but it cannot isolate one terminal.

The design follows four rules. First, every terminal must be available at decision time. Reference tours and reference lengths are never terminal inputs. Second, continuous inputs

use relative values, ranks, or progress ratios when possible. Third, the set covers local edge quality, baseline confidence, search stage, and colony feedback. Fourth, the terminal count must support a controlled Core/Full ablation. We do not assume that Full is better. The Core/Full comparison tests the group-level increment. It does not prove that Core is minimal or identify the causal effect of one terminal.

G. Transition Terminal Set

Many raw ACO values depend on the problem size and iteration. The main terminal set uses bounded relative values. For a vector y and its active mask, define

$$\text{stdrel}(y) = \tanh\left(\frac{y - \text{mean}(y)}{\text{std}(y) + \epsilon}\right). \quad (22)$$

The mean and standard deviation are computed over the last active dimension. Masked entries receive zero.

RTau and REta describe the relative strength of the two baseline ACO signals. BaseConf describes confidence in one candidate. DistRank gives scale-insensitive local geometric order. These four terminals form Core. Entropy describes ambiguity of the complete choice distribution. ConstructProg, ACOProg, and Stagnation describe search context. They enlarge the search space and therefore belong only to Full.

H. Pheromone Terminal Set

Each value in Table V corresponds to one edge in one reinforcement event. The runtime context includes source tours $\pi^{\text{src}}[B, R, n+1]$, their lengths $L^{\text{src}}[B, R]$, colony tours $\pi^{\text{col}}[B, M, n+1]$, and colony lengths $L^{\text{col}}[B, M]$. Each source tour is expanded to endpoints $u, v[B, R, n]$. Every terminal therefore enters the tree as PhField $[B, R, n]$. An event scalar is broadcast over the edge axis. A task scalar is broadcast over both the event and edge axes.

EdgeEta, EdgeTau, and NNRank describe the reinforced edge. SourceQuality describes the route that produced that edge. These four terminals form Core. ColonyFreq, ACOProg, and Stagnation add colony and search-stage context. They belong only to Full.

I. Local-Search-Aware Terminal Extension

The base terminals describe construction and reinforcement. They do not identify which edges survive 2-opt. They also do not describe the gain of the accepted move. The LS-aware extension therefore adds the terminals in Table VI. All values are available during search. No reference tour or optimal label is an input.

Origin, PreFreq, and PostFreq distinguish construction evidence from local-search repair. LSGain assigns move-level credit to the inserted edges. TauHeadroom is useful only when a finite pheromone upper bound is active. These terminals form an LS-aware grammar. The completed experiments do not isolate the causal effect of each added terminal.

Both primitive sets also contain the constants $-1, -0.5, 0, 0.5, 1$. They contain one ephemeral random

TABLE III
RELATION BETWEEN THE PREVIOUS GP-ACO TERMINALS AND THE PRESENT DESIGN

Previous input	Corresponding information in this study	Design reason
Pheromone concentration and distance	RTau, REta, and the retained baseline score s^0	The ACO absolute score structure is retained. The tree observes relative strength inside the current candidate set.
Mean candidate pheromone and mean candidate distance	Within-group normalization of RTau/REta, plus BaseConf and Entropy	Location and scale enter the normalization. The latter terminals directly describe concentration of the baseline choice distribution.
Number of cities and number of unvisited cities	ConstructProg, DistRank, and NNRank	Ratios and ranks replace raw counts. Their ranges do not grow linearly with TSP size.
No pheromone-update-specific inputs	EdgeEta, EdgeTau, ColonyFreq, and SourceQuality	The second tree acts on reinforcement events. It requires edge quality, colony agreement, and source-tour quality.

TABLE IV
RUNTIME INPUTS, CALCULATION, AND LOGICAL TENSOR SHAPE OF TRANSITION TERMINALS

Terminal	Type; native shape $\rightarrow$ tree-input shape	Information read from the ACO state	Calculation and candidate-axis meaning	Profile
RTau	TrField; [B, M, K] $\rightarrow$ [B, M, K]	$\tau[B, n, n]$, current city [B, M], candidate city and feasible mask [B, M, K]	Gather pheromone from the current city to each candidate. Normalize $\log(\max(\tau, \epsilon))$ over the feasible K axis of each task-ant pair, then apply tanh.	Core, Full
REta	TrField; [B, M, K] $\rightarrow$ [B, M, K]	$\eta[B, n, n]$, current city [B, M], candidate city and feasible mask [B, M, K]	Gather the heuristic value for each candidate. Apply log normalization over the feasible K axis and then tanh.	Core, Full
BaseConf	TrField; [B, M, K] $\rightarrow$ [B, M, K]	$\tau, \eta, \alpha, \beta$, feasible mask, and $p^0[B, M, K]$ from $s^0 = \tau^\alpha \eta^\beta$	For feasible count k , compute $\tanh[\log(p^0 + \epsilon) + \log k]$ for each candidate. A uniform distribution maps to zero.	Core, Full
DistRank	TrField; [B, M, K] $\rightarrow$ [B, M, K]	Distance matrix $d[B, n, n]$, current city, candidate city, and feasible mask	Rank candidates by distance within each feasible K set. Map rank r to $1 - 2(r - 1)/(k - 1)$. Use zero when $k = 1$.	Core, Full
Entropy	TrField; [B, M, 1] $\rightarrow$ [B, M, K]	Baseline probability $p^0[B, M, K]$, feasible mask, and feasible count $k[B, M, 1]$	Compute $2H(p^0)/\log k - 1$ over K, then broadcast to all candidates. Use -1 when $k = 1$.	Full
ConstructProg	TrField; [1] $\rightarrow$ [B, M, K]	Current construction step q and city count n	Compute $2q/\max(n - 1, 1) - 1$, then broadcast over B, M, K.	Full
ACOProg	TrField; [1] $\rightarrow$ [B, M, K]	Current ACO iteration t and total iterations T	Compute $2(t - 1)/\max(T - 1, 1) - 1$, then broadcast to all ants and candidates.	Full
Stagnation	TrField; [B, 1, 1] $\rightarrow$ [B, M, K]	Task-level non-improvement count $s_t[B]$ and T	Compute $2\min(s_t/T, 1) - 1$, then broadcast over ants and candidates.	Full

constant sampled from $U[-1, 1]$. An ERC mutation adds Gaussian noise with standard deviation 0.1. The result is clipped to $[-1, 1]$. The constants are also strongly typed terminals. A transition constant returns TrField. A pheromone constant returns PhField. The evaluator broadcasts a constant to $[B, M, K]$ or $[B, R, n]$. There is no untyped scalar terminal that can cross the two roles.

J. Function Sets and Numerical Protection

Table VII gives the two function profiles. All operations are element-wise. F1 extends F0. Protected division is

$$\text{PDIV}(x, y) = \frac{xy}{y^2 + 10^{-6}}. \quad (23)$$

This form is continuous at $y = 0$. After each primitive, NaN is replaced by zero. Positive and negative infinity are replaced by 10 and -10 . Every result is then clipped to $[-10, 10]$. The external residual interface applies another tanh.

Each function is registered twice. For example, transition ADD has signature (TrField, TrField) $\rightarrow$ TrField. Pheromone ADD has signature (PhField, PhField) $\rightarrow$ PhField. One call cannot mix the two types.

Legacy-GP uses ADD, SUB, MUL, NEG, and PDIV1. Its protected division returns x/y when $|y| > 10^{-6}$. It returns

one otherwise. Matched-Replace uses the Full terminals and F1. Its raw transition output becomes a positive score through

$$s_{aij} = \text{softplus}(\text{clip}(u_{aij}, -20, 20)) + \epsilon. \quad (24)$$

F0 can form smooth low-order combinations. F1 adds simple piecewise rules through MIN, MAX, and ABS. We omit exponential, logarithmic, and trigonometric primitives. The terminals already include log transformations and bounded normalization. The residual interface also applies tanh. Adding these functions would enlarge the equivalent-expression space and increase numerical risk. We also omit explicit conditional branches. MIN and MAX provide simple gating and are easier to fuse. The F0/F1 factor tests whether the additional functions are necessary.

K. Evolutionary Search

The initial joint population contains four modes. Ten percent are exact baseline sentinels. Twenty percent activate only the transition tree. Twenty percent activate only the pheromone tree. The remaining 50 percent activate both trees. Active trees use ramped half-and-half initialization with depths from two to four.

Elitism keeps the best ten individuals. The remaining offspring use crossover with probability 0.80, mutation with prob-

TABLE V
RUNTIME INPUTS, CALCULATION, AND LOGICAL TENSOR SHAPE OF PHEROMONE TERMINALS

Terminal	Type; native shape $\rightarrow$ tree-input shape	Information read from the ACO state	Calculation and event-edge meaning	Profile
EdgeEta	PhField; [B, R, n] $\rightarrow$ [B, R, n]	$\eta[B, n, n]$, endpoints $u, v[B, R, n]$, and candidate-neighborhood mean $\log \eta[B, n]$ for each city	Subtract the mean endpoint neighborhood value from edge $\log \eta_{uv}$. Normalize over the n edges of each task-event pair and apply tanh.	Core, Full
EdgeTau	PhField; [B, R, n] $\rightarrow$ [B, R, n]	$\tau[B, n, n]$ and endpoints $u, v[B, R, n]$	Gather τ_{uv} for every source edge. Normalize $\log(\max(\tau_{uv}, \epsilon))$ over the n edges of each event and apply tanh.	Core, Full
NNRank	PhField; [B, R, n] $\rightarrow$ [B, R, n]	Precomputed full nearest-neighbor ranks [B, n, n] and endpoints $u, v[B, R, n]$	Map the directed ranks $u \rightarrow v$ and $v \rightarrow u$ by $1 - 2(r - 1) / \max(n - 2, 1)$, then average the two directions.	Core, Full
ColonyFreq	PhField; [B, R, n] $\rightarrow$ [B, R, n]	Current colony tours $\pi^{\text{col}}[B, M, n + 1]$ and source endpoints u, v	Ignore direction and count how many of the M colony tours contain each source edge. Compute $2c_e/M - 1$.	Full
SourceQuality	PhField; [B, R, 1] $\rightarrow$ [B, R, n]	Source length $L^{\text{src}}[B, R]$ and colony lengths $L^{\text{col}}[B, M]$	Compute colony mean $\bar{L}$ and population standard deviation σ_L over M . Use $\tanh[(\bar{L} - L^{\text{src}})/(\sigma_L + \epsilon)]$, then broadcast over the n edges.	Core, Full
ACOProg	PhField; [1] $\rightarrow$ [B, R, n]	Current ACO iteration t and total iterations T	Compute $2(t - 1) / \max(T - 1, 1) - 1$, then broadcast to all events and edges.	Full
Stagnation	PhField; [B, 1, 1] $\rightarrow$ [B, R, n]	Task-level non-improvement count $s_t[B]$ and T	Compute $2 \min(s_t/T, 1) - 1$, then broadcast over events and edges.	Full

TABLE VI
ADDITIONAL STRONGLY TYPED TERMINALS USED BY THE TSP500 LS-AWARE EXTENSION

Terminal	Output type and logical shape	Runtime input	Calculation and meaning
MutualRank	TrField, [B, M, K]	Full directed nearest-neighbor ranks and current candidate edges	Average the two directed normalized ranks and subtract from one. A value near one means that both endpoints rank each other as close neighbors.
TurnCos	TrField, [B, M, K]	Previous city, current city, candidate coordinates, and feasible mask	Compute the cosine between the incoming and outgoing vectors. The result is clipped to $[-1, 1]$; the first construction step uses zero.
Origin	PhField, [B, R, n]	The source tour before and after 2-opt	Use +1 for a constructed edge retained by 2-opt and -1 for an edge introduced by 2-opt.
LSGain	PhField, [B, R, n]	Accepted 2-opt moves, their four endpoints, the pre-search tour length, and n	For each newly inserted edge, store the gain of its last accepted insertion, $\text{clip}(g/(L^{\text{pre}}/n + \epsilon), 0, 1)$. Retained construction edges use zero.
PreFreq	PhField, [B, R, n]	All M colony tours before 2-opt and each source edge	Compute $2c_e^{\text{pre}}/M - 1$, where c_e^{pre} is the undirected pre-search colony frequency.
PostFreq	PhField, [B, R, n]	All M colony tours after 2-opt and each source edge	Compute $2c_e^{\text{post}}/M - 1$. The terminal describes consensus after local search.
TauHeadroom	PhField, [B, R, n]	$\tau_e, \rho, \tau_{\min}, \tau_{\max}$ after selecting the source edge	Compute $\text{clip}([\tau_{\max} - (1 - \rho)\tau_e]/[\tau_{\max} - \tau_{\min} + \epsilon], 0, 1)$. It measures the remaining MMAS deposit headroom after evaporation.

ability 0.15, and reproduction with probability 0.05. Tournament size is four. Crossover selects one active role and applies typed one-point crossover within that role. Mutation selects one active role. It uses subtree mutation with probability 0.50, node replacement with probability 0.30, and ERC perturbation with probability 0.20. An offspring that violates the depth or node limit is replaced by its valid parent.

Structural hashes avoid repeated evaluation within one generation. The GPU evaluator also groups programs with identical compiled behavior. These operations do not change the fitness.

For a training set $\mathcal{I}$ and ACO seed set $\mathcal{S}$, fitness is

$$F(\theta) = \frac{1}{|\mathcal{I}||\mathcal{S}|} \sum_{i \in \mathcal{I}} \sum_{s \in \mathcal{S}} g_{i,s}(\theta). \quad (25)$$

The objective minimizes the absolute reference gap. It does not minimize a baseline-relative value. It contains no parsimony penalty. Program size is used only to break defined ties. For reporting, we separately define

$$\Delta g = g_{\text{method}} - g_{\text{ACO}}. \quad (26)$$

A negative Δg favors the learned method.

L. LS-Aware Multi-Fidelity and Staged Training

Directly adding 2-opt changes the learning problem. Many different construction policies enter the same local optimum basin. The final gap can therefore have little variance even when their pre-search tours differ. The TSP500 extension uses two paired training scores. The screening score is the mean post-search basin score. The high-fidelity score is

$$F_{\text{high}} = \frac{1}{2} \Delta g_{\text{final}} + \frac{1}{2} \Delta g_{\text{anytime}}. \quad (27)$$

Both terms are relative to a cached ACO+2-opt result evaluated on the same instance and seed. This baseline-relative form is used only for the LS-aware training study. Final reporting still uses the absolute reference gap.

The screen/high schedules are 16/32 instances with 50/100 ACO iterations in generations 1–15, 32/64 with 100/200 iterations in generations 16–35, and 64/128 with 200/500 iterations in generations 36–50. The number of high-fidelity ACO seeds increases from one to two and then three. At

TABLE VII
GP FUNCTION SET AND STRONGLY TYPED INPUT-OUTPUT SIGNATURES

Function	Calculation	Input type	Output type	Profile
ADD	$x + y$	TrField×TrField or PhField×PhField	TrField or PhField	F0, F1
SUB	$x - y$	TrField×TrField or PhField×PhField	TrField or PhField	F0, F1
MUL	xy	TrField×TrField or PhField×PhField	TrField or PhField	F0, F1
PDIV	$xy/(y^2 + 10^{-6})$	TrField×TrField or PhField×PhField	TrField or PhField	F0, F1
NEG	$-x$	TrField or PhField	TrField or PhField	F0, F1
MIN	$\min(x, y)$	TrField×TrField or PhField×PhField	TrField or PhField	F1
MAX	$\max(x, y)$	TrField×TrField or PhField×PhField	TrField or PhField	F1
ABS	$ x $	TrField or PhField	TrField or PhField	F1

Algorithm 1 One RMTGP-ACO simulation

Require: TSP instance, ACO variant, T_{tr} , T_{ph}

- 1: Initialize pheromone and variant-specific search state
- 2: **for** $t = 1, \dots, T$ **do**
- 3: **for** each construction step q **do**
- 4: Build the active candidate set
- 5: Compute baseline score $s^0 = \tau^\alpha \eta^\beta$
- 6: Build only the terminals used by T_{tr}
- 7: Apply the bounded residual in (11)
- 8: Select cities with the original variant rule
- 9: Apply the ACS local update when required
- 10: **end for**
- 11: Compute all tour lengths
- 12: Optionally apply ACOTSP-style 2-opt and record pre/post edge state
- 13: Update best states with the post-search tours
- 14: Build AS, ACS, or MMAS deposit events
- 15: Build only the terminals used by T_{ph}
- 16: Apply the budget residual in (19)
- 17: Apply variant evaporation, bounds, and restart logic
- 18: **end for**
- 19: **return** Best tour; recompute its length in CPU float64

most two structurally identical individuals are kept. The full selection, gate, and final test use 5000 ACO iterations.

IV. EXPERIMENTAL METHODOLOGY

A. Research Questions

Table VIII lists six research questions. Every question has a completed experiment. The evolutionary evidence uses three GP runs and remains pilot evidence.

B. Data

The training pool contains 1,280,000 uniform TSP100 instances. It consists of ten files with 128,000 instances each. A frozen schedule selects 32 different instances per generation. One GP run therefore sees $32 \times 50 = 1,600$ different training

Algorithm 2 GP training and champion selection

Require: Frozen training and validation schedules

- 1: Load or create immutable baseline ACO archives
- 2: Initialize 100 typed two-tree individuals
- 3: **for** $h = 1, \dots, 50$ **do**
- 4: Load 32 scheduled TSP100 training instances
- 5: Use one shared ACO seed for all individuals
- 6: Evaluate unique programs and assign (25)
- 7: Save the top five every five generations
- 8: Create the next population with typed operators
- 9: **end for**
- 10: Screen unique checkpoints on 32 selection instances with one seed
- 11: Re-evaluate the best five with three paired seeds
- 12: Find the minimum mean gap
- 13: Within 0.01 percentage points, select the smaller program
- 14: Audit this candidate on 32 disjoint gate instances
- 15: **return** One selected candidate and one gate-deployed policy

instances. The full pool is much larger than the consumed subset. This design supports independent schedules for additional GP runs.

The validation file contains 1,280 uniform TSP100 instances. Each GP run uses 32 instances for selection and 32 disjoint instances for the non-inferiority gate. The completed test sets contain 1,280 uniform TSP50 instances, 1,280 uniform TSP100 instances, 128 uniform TSP500 instances, and 128 uniform TSP1000 instances. TSP1000 is a locked-model supplementary extrapolation set. It is not used during evolution or model selection.

The registered distribution tests contain 128 clustered TSP500 instances, 128 Gaussian TSP500 instances, and 42 TSPLIB instances with $n \leq 500$ [12]. They are used only for the locked Full-F1 champions. No program is selected again on these partitions. All files are identified by a manifest and a SHA-256 checksum. Train, validation, and test coordinates remain separated. Reference-tour edges are never used as GP terminals.

TABLE VIII
RESEARCH QUESTIONS, PRIMARY COMPARISONS, AND CURRENT STATUS

Question	Research question	Primary comparison	Status
RQ1	Does RMTGP-ACO improve baseline ACO without local search, and does it transfer across sizes and distributions?	Full-F1 versus AS/ACS/MMAS on uniform TSP50–1000 and locked OOD sets	Complete
RQ2	Which conclusions are supported for residual integration, two trees, primitive profiles, pair coadaptation, and expression capacity?	Replacement, one-tree, Core/Full, F0/F1, drop/shuffle, and B31/B62 controls	Complete
RQ3	How do 32, 64, and 128 training instances affect quality and cost?	Paired ACS training-budget experiment with identical nested schedules	Complete
RQ4	Why does 2-opt weaken learning, and which signal changes are justified?	Initial LS study, signal audit, formal TSP100 fitness controls, and TSP500 racing audit	Complete
RQ5	Can LS-aware RMTGP-ACO+2-opt beat ACO+2-opt and approach ACO+3-opt on TSP500?	Three distributions, three GP seeds, three ACO seeds, and 5000 iterations	Complete
RQ6	What acceleration and deployment cost are achieved?	RTX 4000 Ada fused benchmark, RTX A5000 deployment timing, and RTX PRO 5000 Blackwell LS kernels	Complete

Algorithm 3 Three-stage TSP500 LS-aware evolution

Require: 128 frozen training instances, no-LS transition champions

- 1: **for** $h = 1, \dots, 50$ **do**
- 2: **if** $h \leq 10$ **then**
- 3: Set the transition residual to zero; evolve the pheromone tree
- 4: **else if** $h \leq 30$ **then**
- 5: Inject and freeze the matched no-LS transition champion
- 6: **else**
- 7: Jointly evolve both trees; choose PH/TR with probability 0.7/0.3
- 8: **end if**
- 9: Evaluate the population with the registered screen fidelity
- 10: Re-evaluate finalists with the registered high fidelity
- 11: Breed from the high-fidelity ranking
- 12: **end for**
- 13: Select and gate checkpoints on disjoint TSP500 validation instances
- 14: **return** One selected candidate for each GP seed

TABLE IX
DATA USED BY THE TSP100 PILOT

Role	Partition	Instances
Training pool	TSP100 uniform	1,280,000
Per GP run	32×50 TSP100	1,600
Validation	Selection / gate	32/32
Test	TSP50 uniform	1,280
Test	TSP100 uniform	1,280
Test	TSP500 uniform	128
Test	TSP1000 uniform	128
OOD test	TSP500 cluster / Gaussian	128/128
OOD test	TSPLIB, $n \leq 500$	42

The LS-aware study is separate from the TSP100 study. It trains only on TSP500. Each generation draws from a frozen set of 128 uniform instances. Selection and gate use 16 and 48 disjoint uniform TSP500 instances. The final test uses 32

TABLE X
DATA USED BY THE LS-AWARE TSP500 STUDY

Role	Partition	Instances
Training	TSP500 uniform	128
Validation	Selection / gate	16/48
Final test	TSP500 uniform	32
Final test	TSP500 cluster	32
Final test	TSP500 Gaussian	32

instances from each of the uniform, clustered, and Gaussian TSP500 partitions. Each selected candidate is evaluated with three ACO seeds. The same reference tour is used by RMTGP-ACO+2-opt, ACO+2-opt, and ACO+3-opt.

C. Compared Methods and Ablations

Table XI maps each method to its scientific purpose. Full-F1 is the proposed main configuration. Core and Full refer to the terminal sets in Tables IV and V. F0 and F1 refer to Table VII.

RQ2 uses TR-RGP minus Matched-Replace as its primary estimand. Legacy-GP is a secondary methodological baseline. RQ2 also compares Full-F1 with independently trained TR-RGP and PH-RGP. All three methods first use a total node budget of 31. A separate capacity study compares budgets of 31 and 62. RQ2 uses a 2×2 terminal–function factorial design. It estimates the terminal main effect, the function main effect, and their interaction. RQ2 also drops each tree and shuffles independently evolved tree pairs. The post-hoc interventions complement the independent one-tree controls. They do not replace them.

The local-search studies compare four locked policies. *ACO+2-opt* and *ACO+3-opt* use the original ACO rules. *RMTGP(no-LS)+2-opt* applies a program learned without local search and then runs 2-opt. *RMTGP(LS)+2-opt* or *LS-aware v2* learns while 2-opt is active. Only one selected candidate from each GP run enters final testing.

D. ACO and GP Parameters

Table XII gives the common parameters. AS uses $\rho = 0.5$. ACS uses $\rho = 0.1$, $q_0 = 0.9$, and $\xi = 0.1$. MMAS uses

TABLE XI
METHODS, CONTROLLED DIFFERENCES, AND CURRENT RESULT STATUS

Method	Transition integration	Pheromone integration	Terminal/function profile	Main purpose	Status
Baseline ACO	None	None	None	Original AS, ACS, or MMAS	Complete
Legacy-GP	Full replacement	None	Legacy	Prior GP-ACO structure under the current budget	Complete
Matched-Replace	Full replacement	None	Full/F1	Replacement control for RQ2	Complete
TR-RGP	Residual	None	Full/F1	Independent transition-only control	Complete
PH-RGP	None	Budget residual	Full/F1	Independent pheromone-only control	Complete
RMTGP Core-F0	Residual	Budget residual	Core/F0	RQ2 terminal/function control	Complete
RMTGP Core-F1	Residual	Budget residual	Core/F1	RQ2 terminal/function control	Complete
RMTGP Full-F0	Residual	Budget residual	Full/F0	RQ2 terminal/function control	Complete
RMTGP Full-F1	Residual	Budget residual	Full/F1	Proposed main method	Complete
Drop and shuffle	Post-hoc intervention	Post-hoc intervention	Frozen Full/F1 trees	RQ2 coadaptation analysis	Complete

TABLE XII
COMMON EXPERIMENTAL PARAMETERS

Parameter	Value
ACO ants	32
ACO iterations	500
Candidate-list size	20
α, β	1, 2
γ_{tr}, γ_{ph}	1/3, 1/3
Local search	Disabled
GP population	100
GP generations	50
Crossover / mutation / reproduction	0.80/0.15/0.05
Elites / tournament size	10/4
Initial depth / maximum depth	2-4/5
Total effective node budget	31
Checkpoint interval / retained count	5/5
Training instances per generation	32
Selection / gate instances	32/32
Screening / final validation seeds	1/3
Test ACO seeds per instance	3
Independent GP runs per variant	3
CPU threads / process count	16/1
CUDA block threads	32
Search / final score precision	FP32 / FP64

$\rho = 0.02$. Its update period is 25. Its p_{best} value is 0.05. Its branch check period is 100. Its branch parameter is 0.05. Its branch threshold is 1.00001. Its restart stagnation limit is 250. ACS applies synchronous local updates.

The GP root seeds are 1001–1003 for AS, 2001–2003 for ACS, and 3001–3003 for MMAS. Final test seeds are derived from a separate root seed of 9001. They are independent of the GP seeds. All methods within a paired comparison share the same instances and ACO seeds.

E. Champion Selection and Reported Policy

Each generation uses one ACO seed. All individuals share that seed and the same 32 instances. This pairing reduces within-generation noise. The baseline ACO values are pre-computed and stored in immutable archives.

The validation procedure first screens checkpoints with one seed. It then evaluates the five best checkpoints with three seeds. It selects the smallest tree among candidates

TABLE XIII
PARAMETERS SPECIFIC TO THE TSP500 LS-AWARE STUDY

Parameter	Value
ACO ants / final iterations	32/5000
Local search	ACOTSP-style 2-opt
Candidate list / DLB	20/enabled
GP population / generations	100/50
Initial / maximum depth	2-5/7
Shared total node budget	62
Training instances	128
Screen instances	16 $\rightarrow$ 32 $\rightarrow$ 64
High-fidelity instances	32 $\rightarrow$ 64 $\rightarrow$ 128
High-fidelity seeds	1 $\rightarrow$ 2 $\rightarrow$ 3
Selection / gate instances	16/48
Final GP / ACO seeds	3/3
CUDA precision	FP32-fast

within 0.01 percentage points of the best mean gap. Only this selected individual enters the independent gate. The gate accepts a candidate when the one-sided 95% upper bound of its validation delta is at most 0.10 percentage points.

The main scientific analysis uses the selected candidate from each GP run. It retains all three GP runs. It does not select the best GP seed. The gate-deployed policy is a separate operational quantity. It falls back to baseline ACO when a candidate fails the gate.

The TSP500 LS-aware study uses the same distinction. Its scientific test loads each run's `selected_candidate.pkl`, even when the deployment gate would select the baseline fallback. This policy prevents removal of unfavorable GP runs. AS passes the gate in all three runs. ACS and MMAS do not pass it in the completed v2 study.

F. Statistical Analysis

The primary response is the reference gap in (3). We also record worst-decile conditional value at risk, anytime gap area under the curve, best iteration, wall time, and win/tie/loss rates. A win means that the learned method has a smaller gap than its paired baseline.

The three ACO seeds are first averaged within an instance. The main statistical block is then GP-run by instance. We use a paired Wilcoxon signed-rank test [13]. Holm correction is applied within each ACO variant and RQ family [14]. We also use a hierarchical bootstrap [15]. It resamples GP runs, then instances, then ACO seeds. Each interval uses 10,000 replicates.

For the TSP500 LS-aware comparison, the hierarchical bootstrap resamples GP runs, instances, and ACO seeds. The registered success rule against ACO+2-opt requires at least a 10% relative gap reduction, a negative one-sided 95% upper bound, and improvement in all three GP runs. Comparison with ACO+3-opt is reported separately.

Only three GP runs are available in this pilot. This count is too small for a final claim about evolutionary-run uncertainty. The confidence intervals and tests are therefore descriptive. A confirmatory study will use 30 independent GP runs.

G. Evolution Curves and Final-Test Protocol

The training curve does not use the median fitness of the population. For each independent GP run and generation, we first select the individual with minimum fitness. Program size breaks an exact fitness tie. The training point is the mean reference gap of that generation-best individual on the 32 new training instances. Because every generation uses a different batch, short-term changes are not strict estimates of generalization.

The validation point evaluates the same generation-best individual on the fixed 32 selection instances and the fixed screening seed. It is not the population median or the best-so-far validation value. For each ACO variant, the plotted line is the median of the three independent GP runs at the same generation. The band is their minimum and maximum. The baseline line uses the same fixed validation instances and seed.

The final test does not use the last-generation individual directly. The candidate pool contains the top five individuals from every fifth generation and the top five from the last generation. Duplicate structures are removed. Screening, three-seed reevaluation, and the independent gate select one champion for each GP run. All three champions enter the scientific test. The test sets are used only for final measurement.

The TSP500 evolution plot uses the generation champion within each run. It then reports the median and range over the three GP seeds. Validation points are aggregated only when all three runs use the same ACO horizon at that generation. Five early AS validation points are excluded because two recovered checkpoints retained an older 5000-iteration monitor while the third run used 100 or 200 iterations. These monitor values did not breed or select the final candidate. All final tests use the same 5000-iteration protocol.

V. EFFICIENT IMPLEMENTATION

A. Batched Evaluation Architecture

The nested fitness is expensive. One population contains 100 individuals. Each individual invokes ACO on 32 instances. Each ACO invocation constructs 32×500 tours. A direct

Python loop over individuals, instances, ants, and iterations is not practical.

The implementation uses NumPy, Numba, PyTorch, DEAP, and CuPy [11], [16]–[19]. The CPU backend forms a population-by-instance task matrix. Numba compiles the inner ACO loops. The GPU backend goes further. It compiles each typed GP tree to a compact postfix program. It removes structurally and behaviorally duplicate programs. It then forms a program-by-instance task grid.

One CUDA block owns one program–instance task. Thirty-two threads match the 32 ants. The block performs all 500 ACO iterations. It evaluates both GP programs inside the kernel. It also performs candidate fallback, ACS local-update multiplicities, MMAS bounds, and MMAS restart logic. Problem tensors and terminal-independent features remain resident on the device. A counter-based random generator makes task partitioning deterministic.

The GPU searches in FP32. It returns the best tour. The host recomputes that tour length from the original float64 distance matrix. The final fitness and all paper tables therefore use CPU FP64 tour scoring.

B. Parallel Scheduling

One GP run uses one process and one visible GPU. Population-level and instance-level work are fused inside the kernel. This design avoids multiple Python processes competing for one GPU. Independent GP runs are the main multi-GPU unit. They are assigned to different devices. This campaign mode maximizes aggregate throughput.

A dual-shard mode can split one generation over two GPUs. It reduces the latency of one run. Its scaling is weaker because each shard is smaller. The registered scheduler therefore uses independent runs when enough runs are available. CPU16 remains the deterministic fallback and the FP64 audit backend.

C. Matrix-Accelerated Local Search

The LS-aware CUDA backend flattens program–instance pairs into one task matrix. Each task contains 32 ants. Construction, 2-opt, and pheromone update operate on large task batches instead of Python loops over individuals or instances. The resident problem cache stores coordinates, distance matrices, candidate lists, ranks, and terminal-independent features. Baseline ACO results are computed once and reused by all paired evaluations.

A dense edge-gain tensor with shape $[B, M, n, n]$ would require about 62 GiB for the registered TSP500 batch. The implementation instead stores the two adjacent edges of each city as $[B, M, n, 2]$. An accepted 2-opt move updates only four endpoints. The storage is $O(n)$ per tour. With eight tours per block at TSP500, it uses 32 KiB of shared memory. A warp evaluates the 20 candidate edges of 2-opt in parallel. The 3-opt kernel evaluates the 20^2 candidate pairs with one 512-thread block per tour.

PreFreq and PostFreq use byte matrices $[B, n, n]$ and parallel scatter. Programs are split by terminal semantics. A program that does not read Origin, PreFreq, or LSGain does not allocate their heavy state. Generated CUDA modules

TABLE XIV
FORMAL ACCELERATOR BENCHMARK

Mode	Time (s)	Tours/s	CPU16 ratio
CPU8	397.422	101,776	0.766
CPU16	304.325	132,911	1.000
GPU0	20.807	1,943,957	14.626
GPU1	20.838	1,941,110	14.605
Dual shard	12.776	3,165,928	23.820
Campaign [†]	21.101	3,833,840	28.9

[†]Two concurrent workloads; ratio uses aggregate throughput.

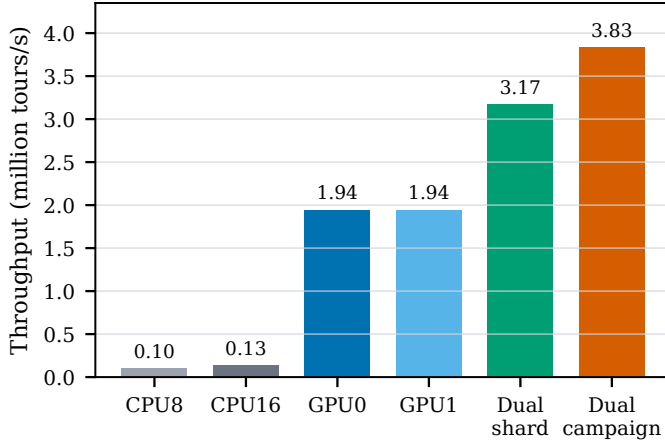


Fig. 2. Throughput on the formal accelerator benchmark. Campaign throughput is aggregated over two concurrent workloads.

use a bounded LRU cache. The task scheduler estimates available memory, chunks the task matrix, and halves the chunk after an out-of-memory failure. Final tests distribute independent partition-variant-seed shards over the available GPUs.

D. Formal Acceleration Benchmark

The formal benchmark used an Intel Xeon W-2245 CPU and two 20-GiB NVIDIA RTX 4000 Ada Generation GPUs. The workload used ACS, 100 requested individuals, 79 behaviorally unique programs, 16 TSP50 instances, 16 TSP100 instances, 32 ants, and 500 ACO iterations. One workload constructed 40,448,000 tours. Each mode used one cold run and three warm runs. Table XIV reports the warm median.

Single-run dual-GPU scaling is $20.807/12.776 = 1.629$. Concurrent campaign scaling is $2(20.807)/21.101 = 1.972$. The first value is below the registered 1.7 scaling gate. The second value is above the registered 1.8 campaign gate. This result supports task-level multi-GPU scheduling.

Average GPU utilization was about 99%. Peak device memory was about 286 MiB. About 99.8% of warm time was inside the CUDA kernel. The workload was compute-bound. Additional memory caching is therefore unlikely to be the main source of future speedup.

The numerical quality audit used TSP50 and TSP100. It used 128 instances per size and three paired ACO seeds. Let $\Delta_{\text{num}} = g_{\text{GPU}} - g_{\text{CPU}}$. The pooled mean was -0.003445 percentage points. The one-sided 95% upper bound was 0.000868

TABLE XV
OBSERVED GENERATION TIME ACROSS THREE GP RUNS

Variant	Mean $\pm$ SD (s)	Run range (s)
AS	34.50 $\pm$ 2.97	31.13–36.78
ACS	26.88 $\pm$ 4.43	21.97–30.57
MMAS	22.14 $\pm$ 3.34	18.31–24.41

percentage points. This value is below the registered 0.10 threshold.

The observed mean generation times are shown in Table XV. They include all unique population evaluations for one generation. They do not include final test evaluation.

VI. RESULTS AND ANALYSIS

A. RQ1: Effectiveness and Generalization Without Local Search

Table XVI reports one selected candidate from each GP run. Negative Δg favors RMTGP-ACO.

All 12 mean differences and intervals are below zero. The reduction ranges from 0.0736 percentage points for ACS on TSP50 to 7.8461 percentage points for AS on TSP500. The three ACO variants also improve on the unseen TSP500 and TSP1000 sizes. The curves are not monotonic. A new instance batch is used at every generation. The selected checkpoint can therefore differ from the last generation.

All nine OOD means are negative. The TSPLIB win rates are lower than the synthetic OOD rates. They are 82.5% for AS, 77.8% for ACS, and 93.7% for MMAS. The programs are not reselected on any OOD partition.

Answer to RQ1: The pilot supports improvement over the original ACO and transfer across sizes. The locked OOD means also favor Full-F1. This result does not identify which design choice causes the improvement.

B. RQ2: Controlled Design Choices

Full-F1 has a lower gap than retrained Legacy-GP in all six contexts. The paired reduction ranges from 0.2918 to 12.9177 percentage points. Every interval is below zero. This is an overall comparison between two representation bundles. It does not isolate the residual interface, the terminal set, or the two-tree architecture.

Residual integration is better than matched replacement for AS. Matched replacement is better for ACS. The two MMAS intervals cross zero. Residual integration is therefore not universally better. This also explains why similarity between the bounded residual and MMAS pheromone bounds is not a sufficient mechanism claim. They constrain different variables.

At 31 nodes, Full-F1 does not beat both one-tree controls in any context. The factorial study gives no interval that supports a universal Full-terminal benefit. It also gives no universal F1-function benefit. The drop interventions show that both trees affect many selected champions. The shuffle interventions do not show stable within-run pair coadaptation. Full-F1 is effective, but the complete grammar is over-specified in some contexts.

TABLE XVI

SELECTED-CANDIDATE RESULTS ON UNIFORM TEST SETS WITHOUT LOCAL SEARCH. CI IS THE HIERARCHICAL BOOTSTRAP 95% INTERVAL OF Δg . W/T/L IS REPORTED IN PERCENT.

ACO	n	Baseline gap (%)	Full-F1 gap (%)	Δg (pp)	95% CI (pp)	W/T/L
AS	50	1.8803	1.7479	-0.1324	[-0.2076, -0.0718]	54.7/2.8/42.4
AS	100	5.8952	4.2154	-1.6799	[-1.9680, -1.4739]	92.1/0.0/7.9
AS	500	22.9874	15.1413	-7.8461	[-8.3979, -7.3727]	100.0/0.0/0.0
AS	1000	30.4224	23.1088	-7.3135	[-8.5940, -6.1418]	100.0/0.0/0.0
ACS	50	0.7326	0.6589	-0.0736	[-0.1050, -0.0432]	46.6/17.6/35.9
ACS	100	1.9331	1.6412	-0.2918	[-0.3954, -0.2091]	64.7/0.0/35.3
ACS	500	16.1641	12.9399	-3.2242	[-4.5698, -1.3932]	93.8/0.0/6.2
ACS	1000	21.6293	19.2041	-2.4252	[-3.3961, -0.8317]	94.8/0.0/5.2
MMAS	50	0.5941	0.4727	-0.1215	[-0.1549, -0.0820]	52.3/22.4/25.2
MMAS	100	4.1088	1.8106	-2.2982	[-2.4240, -2.1223]	99.2/0.0/0.8
MMAS	500	20.6291	14.1769	-6.4522	[-8.1652, -4.7067]	100.0/0.0/0.0
MMAS	1000	25.4885	18.8571	-6.6314	[-8.3746, -4.8120]	100.0/0.0/0.0

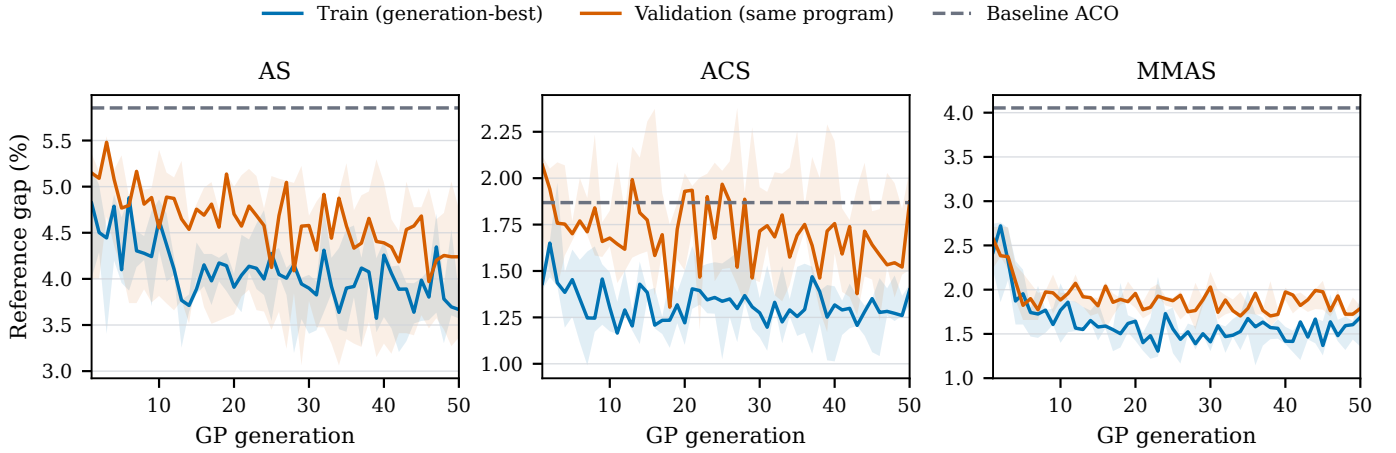


Fig. 3. TSP100 curves without local search. Each run contributes its generation champion. Lines are medians over three GP runs. Bands are the minimum and maximum.

TABLE XVII

DESCRIPTIVE RESULTS ON LOCKED DISTRIBUTION-SHIFT TESTS. THESE TESTS USE THE SAME TSP100-TRAINED FULL-F1 CHAMPIONS.

ACO	Test	Baseline gap (%)	Full-F1 gap (%)	Δg (pp)	W/T/L
AS	TSP500-C	26.0627	19.7383	-6.3245	100.0/0.0/0.0
AS	TSP500-G	20.0986	14.0184	-6.0802	100.0/0.0/0.0
AS	TSPLIB $\leq$ 500	10.0277	7.6313	-2.3964	82.5/0.0/17.5
ACS	TSP500-C	16.3527	13.8345	-2.5182	90.6/0.0/9.4
ACS	TSP500-G	14.8973	12.6798	-2.2175	88.3/0.0/11.7
ACS	TSPLIB $\leq$ 500	4.5376	3.5976	-0.9399	77.8/0.0/22.2
MMAS	TSP500-C	22.7375	16.5961	-6.1414	100.0/0.0/0.0
MMAS	TSP500-G	17.6609	12.1975	-5.4635	100.0/0.0/0.0
MMAS	TSPLIB $\leq$ 500	8.3755	5.0336	-3.3419	93.7/2.4/4.0

The completed capacity study contains 18 within-architecture capacity contrasts. Only one B62-minus-B31 interval is fully below zero. Ten point estimates are positive. The selected expressions also rarely reach either node limit. For the two-tree method, B62 improves the ACS TSP100 mean by 0.1049 percentage points. The other five B62-minus-B31 intervals cross zero. Extra capacity is not the main bottleneck. The B31 model is the more parsimonious default.

Answer to RQ2: The evidence supports the residual interface for AS, but not as a universal rule. The complete current representation outperforms the retrained prior-study

representation in all six matched contexts. This comparison is not a component-level causal result. It does not support a general two-tree, Full-terminal, or F1-function advantage. Both semantic roles can matter. The benefit depends on the ACO variant and test context.

C. RQ3: Training-Instance Budget

All 12 budget-test means improve on baseline ACS. The 64-instance point is best on TSP50 and TSP100. The 128-instance point is best on TSP500 and TSP1000. However, only three

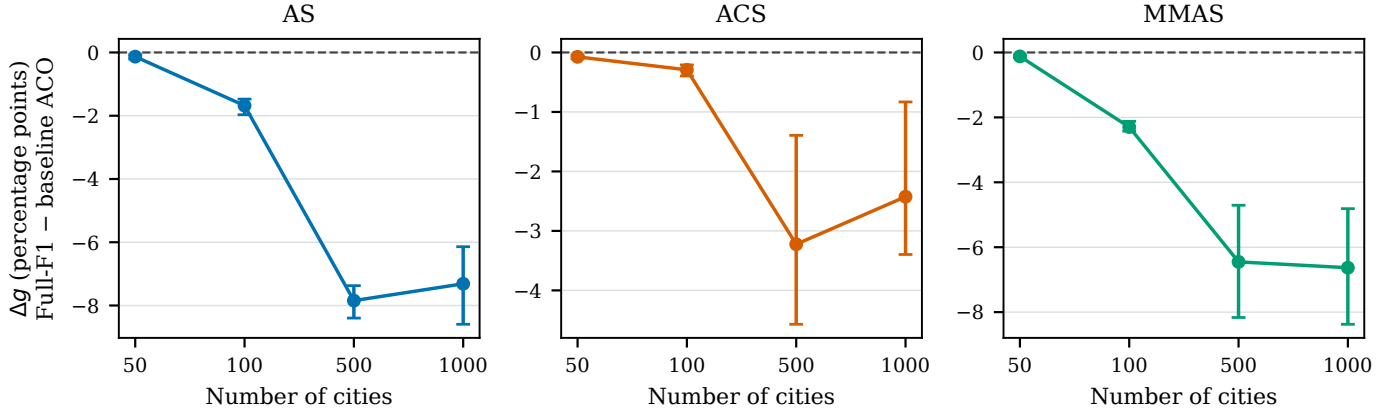


Fig. 4. Full-F1 minus baseline ACO gap. Error bars are hierarchical bootstrap 95% intervals.

TABLE XVIII

PRIMARY CONTROLLED CONTRASTS AT THE 31-NODE BUDGET. EACH CELL IS ESTIMATE [95% HIERARCHICAL BOOTSTRAP INTERVAL] IN PERCENTAGE POINTS. NEGATIVE VALUES FAVOR THE FIRST NAMED METHOD.

ACO	Test	TR-RGP — replacement	Full-F1 — TR-RGP	Full-F1 — PH-RGP
AS	TSP100-U	-0.606 [-0.853, -0.413]	-0.525 [-0.876, -0.265]	-0.068 [-0.230, +0.100]
AS	TSP500-U	-2.558 [-3.119, -1.720]	-0.994 [-1.580, -0.153]	+0.795 [+0.597, +1.007]
ACS	TSP100-U	+0.311 [+0.216, +0.436]	-0.006 [-0.179, +0.113]	-0.263 [-0.393, -0.167]
ACS	TSP500-U	+5.430 [+3.726, +6.602]	+0.304 [-1.843, +2.336]	-2.950 [-4.575, -1.219]
MMAS	TSP100-U	-0.679 [-1.224, +0.291]	-0.056 [-0.182, +0.144]	-0.828 [-0.970, -0.659]
MMAS	TSP500-U	-1.706 [-3.790, +0.452]	-0.810 [-2.601, +1.187]	-1.162 [-3.075, +0.731]

TABLE XIX

MATCHED-BUDGET COMPARISON WITH THE REPRESENTATION FROM THE PRIOR STUDY. THE FIRST TWO NUMERIC COLUMNS ARE MEAN OPTIMALITY GAPS. THE LAST COLUMN IS THE PAIRED ESTIMATE [95% HIERARCHICAL BOOTSTRAP INTERVAL] IN PERCENTAGE POINTS. NEGATIVE VALUES FAVOR FULL-F1.

ACO	Test	Legacy-GP gap (%)	Full-F1 gap (%)	Full-F1 — Legacy-GP (pp)
AS	TSP100-U	5.8952	4.2154	-1.680 [-1.967, -1.473]
AS	TSP500-U	22.9874	15.1413	-7.846 [-8.423, -7.372]
ACS	TSP100-U	1.9331	1.6412	-0.292 [-0.395, -0.208]
ACS	TSP500-U	16.1641	12.9399	-3.224 [-4.576, -1.378]
MMAS	TSP100-U	3.7469	1.8106	-1.936 [-2.427, -1.458]
MMAS	TSP500-U	27.0946	14.1769	-12.918 [-20.080, -5.943]

GP seeds are available. The direction is not consistent over all seeds. The result is a scale-dependent trend, not a significant budget main effect. Increasing the batch from 32 to 128 raises throughput from 6.44 to 9.36 million tours/s. End-to-end time increases by about a factor of two, not four.

Answer to RQ3: Larger batches use the GPU more efficiently. A 64-instance batch gives the best TSP100 cost-quality point. A 128-instance batch gives the best large-size point estimates. The three-seed instance study does not prove that more training data alone causes better GP solutions.

D. RQ4: Learning Signal Under 2-opt

The first local-search study reveals two problems. TSP100 is close to saturation after 2-opt. Training with 2-opt is also not consistently better than training without it. On TSP500, AS and ACS improve on ACO+2-opt, but remain worse than ACO+3-opt. MMAS does not improve on ACO+2-opt.

The post-search variance is only 3.47%, 1.08%, and 1.46% of the pre-search variance for AS, ACS, and MMAS. Final-

gap SNR is at most 0.146. The post-basin signal is much stronger for AS and modestly stronger for ACS. About 9–21% of final edges are introduced by 2-opt. Only about one third of construction differences survive. These measurements motivate basin and anytime fitness. They also motivate Origin, LSGain, PreFreq, and PostFreq. They do not prove that any one terminal is necessary.

The Anytime+Final control passes all three AS deployment gates and reduces the AS validation gap by 15.4%. It gives no corresponding benefit for ACS or MMAS. The Basin+Final control passes no gate. This result shows that a denser fitness alone is not sufficient. It also supports moving the next training study away from saturated TSP100.

The screen has many nonzero responses, but its SNR is below one for every variant. AS has adequate ranking agreement. ACS and MMAS also miss the registered recall and correlation gates. Formal racing therefore does not start under that protocol. The final v2 procedure replaces the small fixed screen with increasing high-fidelity batches up to 128

TABLE XX
ACS INSTANCE-BUDGET TEST. EACH CELL IS MEAN GAP / Δg IN PERCENTAGE POINTS.

Test	Baseline	32 instances	64 instances	128 instances
TSP500-U	0.7005	0.6544 / -0.0461	0.6501 / -0.0504	0.6757 / -0.0248
TSP100-U	1.9358	1.6106 / -0.3252	1.5977 / -0.3381	1.6258 / -0.3100
TSP500-U	16.0822	12.6558 / -3.4265	12.4316 / -3.6506	12.2874 / -3.7948
TSP1000-U	21.6125	19.1227 / -2.4898	18.8222 / -2.7903	18.7391 / -2.8734

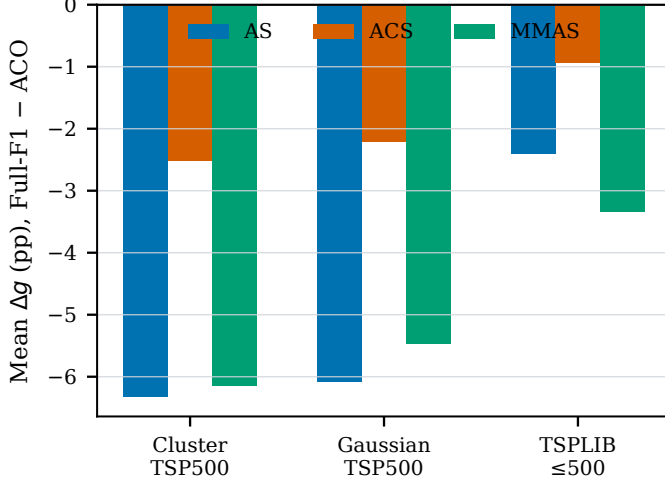


Fig. 5. Mean gap differences on cluster, Gaussian, and TSPLIB tests. Negative values favor Full-F1.

TABLE XXI
SINGLE-GPU ACS TRAINING EFFICIENCY OVER THREE GP SEEDS. TIME IS MEAN $\pm$ SAMPLE STANDARD DEVIATION.

Instances	Gen. time (s)	End-to-end (min)	M tours/s
32	7.63 $\pm$ 1.48	7.71 $\pm$ 1.24	6.44 $\pm$ 0.61
64	10.43 $\pm$ 2.68	10.04 $\pm$ 2.26	7.57 $\pm$ 0.32
128	17.21 $\pm$ 2.56	15.69 $\pm$ 2.15	9.36 $\pm$ 0.08

instances and three seeds.

Answer to RQ4: 2-opt compresses the final-gap signal and removes many construction differences. Final gap alone is a weak training target. Basin/anytime information is justified by the audit. The completed controls show that fitness replacement alone is insufficient. The failed small-screen gate supports the larger staged high-fidelity design. The terminal-level causal question remains open.

E. RQ5: LS-Aware TSP500 Quality and Runtime

AS satisfies the registered success rule against ACO+2-opt on all three distributions. Its reductions are 1.7109, 0.9735, and 1.3900 percentage points. It also beats ACO+3-opt on uniform and Gaussian TSP500. It is 0.5839 percentage points worse on clustered TSP500. All three AS GP runs improve on ACO+2-opt.

ACS has a small uniform improvement, but its one-sided upper bound crosses zero and the relative reduction is below 10%. Its cluster and Gaussian means are slightly worse than

TABLE XXII
INITIAL TSP500 LOCAL-SEARCH STUDY. VALUES ARE MEAN REFERENCE GAP (%).

ACO	ACO+2opt	RMTGP(no-LS)+2opt	RMTGP(LS)+2opt	ACO+3opt
AS	3.0117	2.6182	2.6609	1.8274
ACS	0.3114	0.2580	0.2588	0.1989
MMAS	0.0867	0.0939	0.0924	0.0304

ACO+2-opt. MMAS is worse on uniform and differs little on the other two distributions. Neither ACS nor MMAS passes the registered gate.

For AS, the mean RMTGP+2-opt runtime is close to the ACO+3-opt runtime. It is faster on the clustered set, equal on the uniform set, and slightly slower on the Gaussian set. MMAS RMTGP+2-opt is 1.97–2.29 times faster than ACO+3-opt, but its quality is worse. Runtime savings and quality gains therefore must be reported together.

Answer to RQ5: The LS-aware method is successful for AS. It consistently beats ACO+2-opt and sometimes beats ACO+3-opt. It does not support the same claim for ACS or MMAS. A claim that RMTGP+2-opt reaches ACO+3-opt is therefore valid only for specific AS distributions.

F. RQ6: Computational Efficiency

The fused evaluator gives a 14.626-fold single-GPU speedup over CPU16. Two-GPU sharding gives 23.820-fold speedup. Two independent concurrent runs give 1.972-fold campaign scaling. These values apply only to the registered RTX 4000 Ada workload.

The increasing fidelity raises the late-stage generation cost. Mean end-to-end time is 14.83 hours for AS, 2.92 hours for ACS, and 2.63 hours for MMAS. AS has 32 deposit events per iteration, while ACS and MMAS have one. Program-dependent semantic paths also differ. The timing does not isolate these causes. The nine runs consume 61.15 recorded GPU-hours in total. The selected-candidate-only final test contains 27 independent shards. Two-GPU scheduling completes it in 3.34 wall-hours, including the latency audit.

Eight 2-opt warps per block improve the selected launch by 1.045 times. The 512-thread 3-opt kernel improves the old warp kernel by 1.909 times on TSP100 and 3.511 times on TSP500. The compact edge-LSGain path is 25.64 times faster than the old per-edge loop. Matrix frequency terminals add less than 1% in the isolated semantic benchmark. The heavier LSGain path still costs 38.4% more than the light path, which justifies semantic sharding.

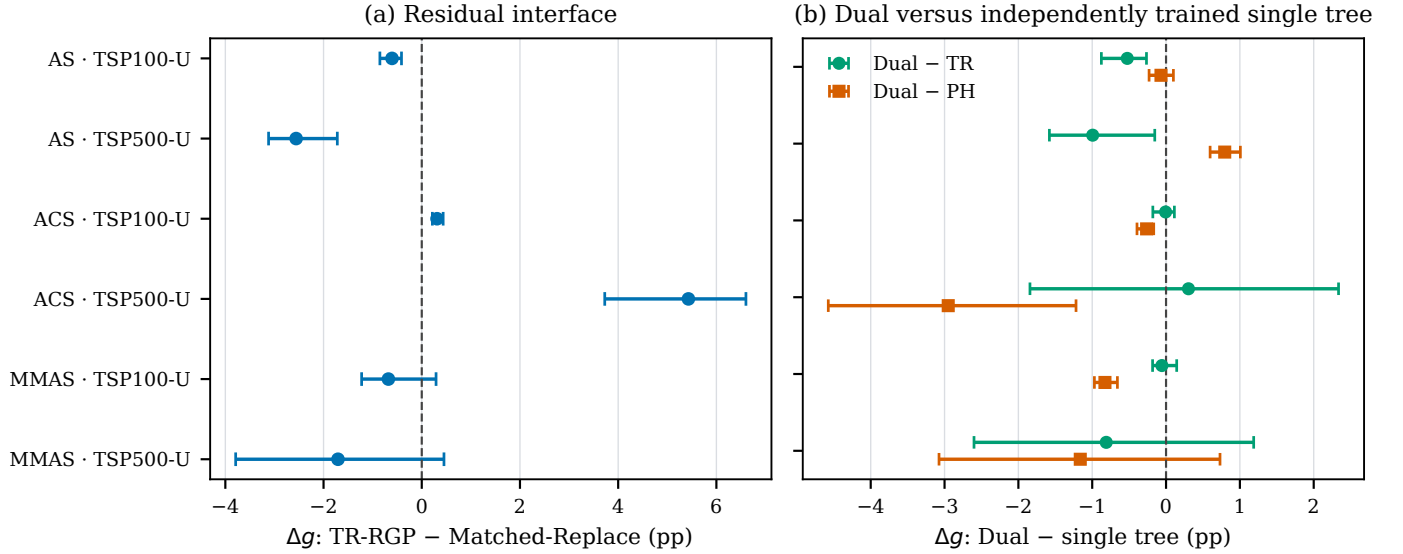


Fig. 6. Primary residual and architecture contrasts. Negative values favor residual integration or the two-tree method, respectively.

TABLE XXIII

TSP100 2-OPT SIGNAL AUDIT AT 500 ACO ITERATIONS. COMPRESSION IS POST/PRE PROGRAM VARIANCE. SNR IS THE BETWEEN-PROGRAM TO WITHIN-SEED VARIANCE RATIO. EDGE COLUMNS ARE PERCENTAGES.

ACO	Baseline gap	Compression	Spearman	Final SNR	Basin SNR	Introduced edges	Difference survival
AS	0.2519	0.0347	0.940	0.100	35.811	21.1	33.5
ACS	0.1103	0.0108	0.660	0.113	1.239	9.4	35.4
MMAS	0.0270	0.0146	0.811	0.146	0.301	9.5	34.3

TABLE XXIV

COMPLETED TSP100 FITNESS CONTROLS ON THE INDEPENDENT 512-INSTANCE GATE. VALUES ARE MEANS OVER THREE SELECTED GP CANDIDATES. PASS REPORTS THE DEPLOYMENT GATE, NOT A FINAL-TEST SUCCESS COUNT.

ACO	Fitness	Baseline gap	Candidate gap	Δg	Gap reduction (%)	Pass
AS	Anytime+Final	0.1301	0.1100	-0.0201	+15.4	3/3
AS	Basin+Final	0.1301	0.1341	+0.0040	-3.1	0/3
ACS	Anytime+Final	0.0448	0.0448	+0.0000	+0.0	0/3
ACS	Basin+Final	0.0448	0.0447	-0.0001	+0.2	0/3
MMAS	Anytime+Final	0.0173	0.0174	+0.0001	-0.4	0/3
MMAS	Basin+Final	0.0173	0.0186	+0.0013	-7.4	0/3

Answer to RQ6: Program–instance fusion makes the nested GP–ACO workload practical. Large instance batches improve occupancy. Compact LS state, semantic sharding, resident caches, and multi-GPU shard scheduling are required for the TSP500 study. The measured full study costs 61.15 GPU-hours for training and 3.34 two-GPU wall-hours for final testing. Hardware-specific speedups are not transferred across GPU models.

VII. LIMITATIONS AND THREATS TO VALIDITY

Every evolutionary condition has only three GP runs. Hierarchical bootstrap intervals contain many instances and ACO seeds, but they cannot replace additional evolutionary runs. The ablation, capacity, and training-budget conclusions are therefore pilot evidence.

The no-LS study trains on uniform TSP100. The LS-aware study trains on uniform TSP500. Distribution transfer is

evaluated on two synthetic distributions and a small TSPLIB subset. The method is implemented only for AS, synchronous ACS, and MMAS with the reported parameters.

The LS-aware validation curves contain a recovery artifact. Five early AS monitor generations used different ACO horizons across seeds. They are excluded from the pooled validation curve. They did not affect breeding or final-test evaluation. All final comparisons use 5000 iterations and selected candidates only.

The latest LS-aware grammar changes several factors together: training size, capacity, terminals, fitness, and staged evolution. Its final result cannot assign causality to LSGain, any other single terminal, or the staged schedule. The ACS and MMAS failures also show that the extension is not universally effective.

Reference tours are valid but do not all have attached global

TABLE XXV
STRICT TSP500 RACING AUDIT AT THE 16-INSTANCE, 500-ITERATION SCREEN. THE REGISTERED GATES ARE $\text{SNR} \geq 1$, $\text{RECALL} \geq 0.8$, AND $\text{SPEARMAN} \geq 0.7$, TOGETHER WITH NONZERO-RESPONSE CHECKS.

ACO	Final nonzero	Anytime nonzero	Combined SNR	Top-32 recall	Spearman	Pass
AS	0.952	0.952	0.651	0.938	0.959	No
ACS	0.890	0.906	0.278	0.719	0.640	No
MMAS	0.813	0.844	0.290	0.719	0.578	No

TABLE XXVI
FINAL TSP500 LS-AWARE TEST. U95 IS THE ONE-SIDED 95% UPPER BOUND VERSUS ACO+2-OPT. THE LAST COLUMN REPORTS REGISTERED SUCCESS VERSUS ACO+2-OPT / SUPERIORITY VERSUS ACO+3-OPT.

ACO	Distribution	ACO+2opt gap	RMTGP+2opt gap	Δ_2	U95 ₂	ACO+3opt gap	Δ_3	Success ₂ /Better ₃
AS	Uniform	3.0662	1.3554	-1.7109	-1.6368	1.7908	-0.4355	Yes/Yes
AS	Cluster	3.0344	2.0609	-0.9735	-0.8820	1.4769	+0.5839	Yes/No
AS	Gaussian	2.7789	1.3889	-1.3900	-1.3082	1.7618	-0.3728	Yes/Yes
ACS	Uniform	0.3221	0.2951	-0.0270	+0.0056	0.1846	+0.1105	No/No
ACS	Cluster	0.2558	0.2593	+0.0035	+0.0498	0.1148	+0.1444	No/No
ACS	Gaussian	0.2891	0.2937	+0.0046	+0.0364	0.2082	+0.0855	No/No
MMAS	Uniform	0.0808	0.1021	+0.0213	+0.0333	0.0367	+0.0654	No/No
MMAS	Cluster	0.0682	0.0666	-0.0016	+0.0103	0.0202	+0.0464	No/No
MMAS	Gaussian	0.1382	0.1341	-0.0041	+0.0078	0.0606	+0.0735	No/No

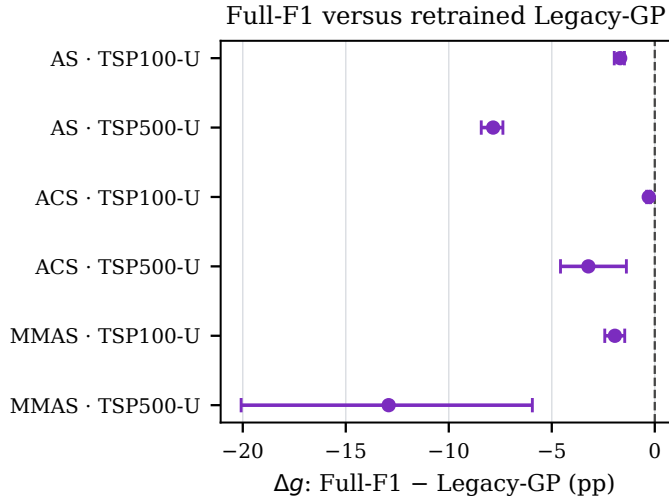


Fig. 7. Paired difference between Full-F1 and retrained Legacy-GP. All intervals are below zero.

TABLE XXVII
DESCRIPTIVE MEAN OVER THE THREE TSP500 DISTRIBUTIONS

ACO	ACO2	RMTGP2	ACO3	Δ_2	Δ_3	Gap reduction (%)
AS	2.9599	1.6017	1.6765	-1.3581	-0.0748	+45.9
ACS	0.2890	0.2827	0.1692	-0.0063	+0.1135	+2.2
MMAS	0.0957	0.1010	0.0392	+0.0052	+0.0618	-5.5

optimality certificates. The term reference gap is therefore used instead of optimality gap. GPU search uses FP32-fast, while final tour length is recomputed in FP64. Discrete ACO trajectories need not be bitwise identical across backends.

Training throughput and single-program latency measure different workloads. The RTX 4000 Ada, RTX A5000, and

RTX PRO 5000 Blackwell numbers must not be combined as one hardware comparison.

VIII. CONCLUSION

RMTGP-ACO evolves two strongly typed residual programs. The transition tree adjusts the original desirability. The pheromone tree redistributes the original deposit budget. Zero output recovers baseline ACO.

Without local search, Full-F1 improves the mean gap in all 12 uniform variant-size comparisons. It also outperforms retrained Legacy-GP in all six matched TSP100/TSP500 contexts. This second result compares complete representations, not isolated components. The detailed controls qualify this result. Residual integration is not always better than replacement. Two trees do not always beat both single-tree controls. The richer terminal and function profiles are not always useful. Increasing the total node budget from 31 to 62 is also not a general remedy.

The 2-opt audit identifies a separate problem. Local search compresses the final-gap signal. The LS-aware extension therefore uses edge-origin and move information, multi-fidelity fitness, and staged evolution on TSP500. This extension is effective for AS. It beats ACO+2-opt on all three tested distributions and beats ACO+3-opt on two. The same conclusion does not hold for ACS or MMAS.

The CUDA implementation enables these nested experiments. Its main improvements are program-instance fusion, compact local-search state, generated GP kernels, semantic sharding, baseline caching, and multi-GPU task scheduling. Future work should increase the number of GP runs and isolate the individual LS-aware design factors.

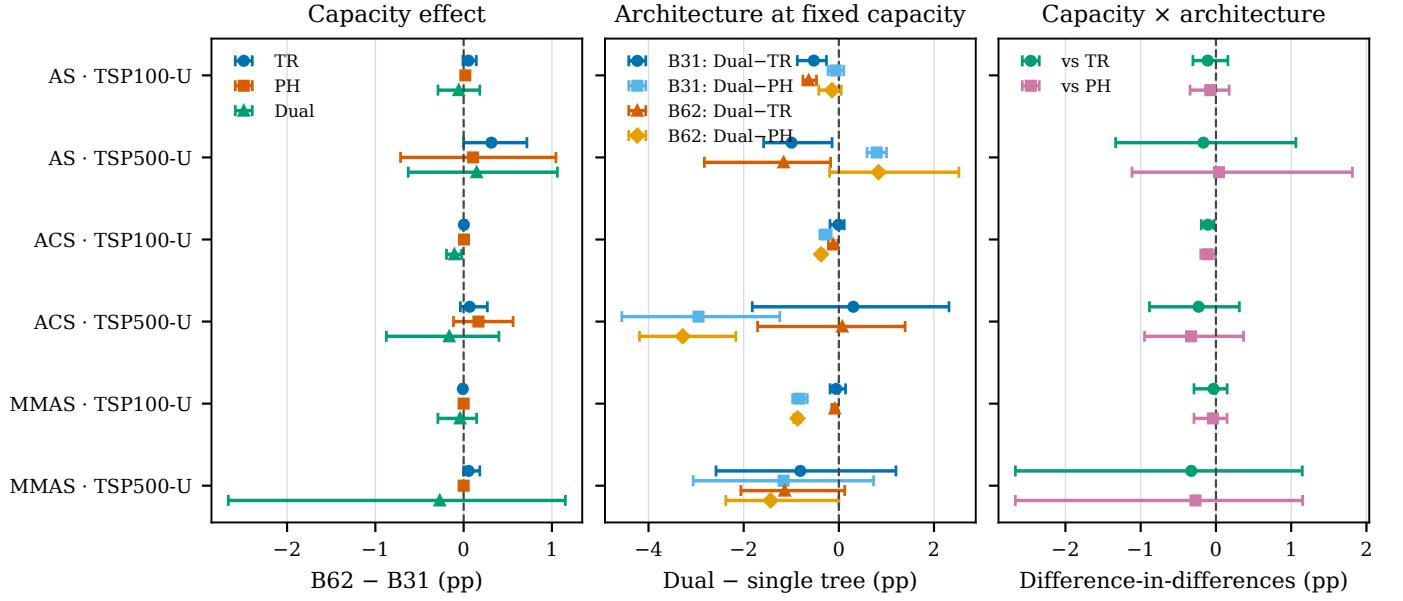


Fig. 8. Capacity and architecture contrasts at total budgets of 31 and 62 nodes. Negative values favor the first term in each panel.

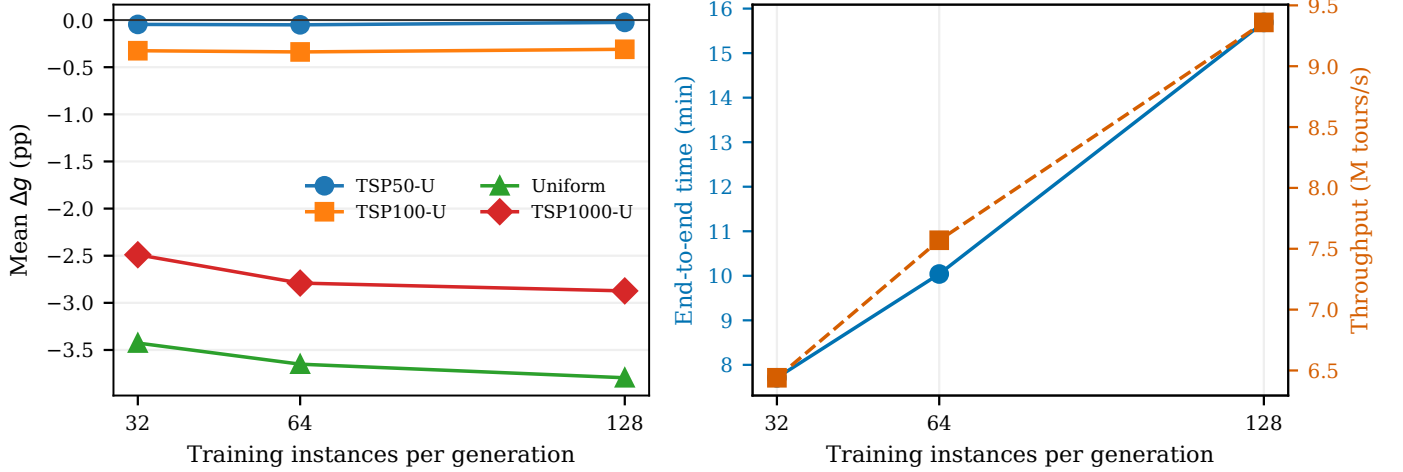


Fig. 9. Quality, end-to-end time, and throughput for 32, 64, and 128 training instances per generation.

APPENDIX A FULL CONTROLLED-ABLATION TABLES

APPENDIX B LOCAL-SEARCH DEVELOPMENT RESULTS

The signal pilots compare final-gap, basin, and combined fitness. They also probe residual radii and the *Origin* terminal. They run for three generations. They are used to select the development direction. They are not confirmatory terminal ablations. The failed strict TSP500 racing gate is likewise treated as diagnostic evidence. It motivated 128 training instances, longer horizons, and the final v2 protocol. No incomplete shard enters a main table.

APPENDIX C TSP500 RUNTIME AND SELECTED PROGRAMS

APPENDIX D ADDITIONAL ENGINEERING MEASUREMENTS

The semantic benchmark evaluates 65 programs on 32 TSP500 instances for ten ACO iterations. *EdgeEta*, *PreFreq*, and *PostFreq* take 1.602, 1.606, and 1.613 seconds. The compact *edge-LSGain* path takes 2.217 seconds. All generated assets record their input paths and SHA-256 values in `paper_assets/generated_from.json`.

REFERENCES

- [1] M. Dorigo, V. Maniezzo, and A. Colomi, "Ant system: Optimization by a colony of cooperating agents," *IEEE Transactions on Systems, Man, and Cybernetics, Part B*, vol. 26, no. 1, pp. 29–41, 1996.
- [2] M. Dorigo and T. Stützle, *Ant Colony Optimization*. Cambridge, MA, USA: MIT Press, 2004.

TABLE XXVIII

COMPLETED TSP500 LS-AWARE TRAINING TIME. VALUES ARE MEAN $\pm$ SAMPLE STANDARD DEVIATION OVER THREE GP RUNS. STAGE COLUMNS REPORT SECONDS PER GENERATION.

ACO	Total (h)	G1–15	G16–35	G36–50	Late M tours/s
AS	14.83 $\pm$ 5.61	117.1 $\pm$ 66.3	454.1 $\pm$ 283.6	2837.3 $\pm$ 1677.6	0.14 $\pm$ 0.07
ACS	2.92 $\pm$ 0.60	24.9 $\pm$ 0.4	80.6 $\pm$ 4.4	569.5 $\pm$ 146.4	0.53 $\pm$ 0.12
MMAS	2.63 $\pm$ 0.20	35.7 $\pm$ 1.6	100.2 $\pm$ 5.3	460.9 $\pm$ 54.2	0.63 $\pm$ 0.06

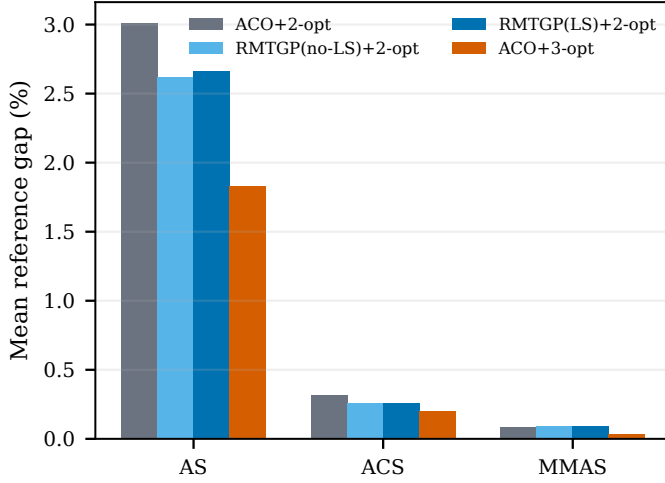


Fig. 10. Initial TSP500 local-search comparison using final selected candidates.

TABLE XXIX
COMPLETED TSP500 CAMPAIGN ACCOUNTING

Phase	Work	Time	Accounting
Training	9 GP runs	61.15 GPU-h	Sum of recorded run wall times
Final test	27 shards	3.34 wall-h	2 GPUs; includes latency audit

TABLE XXX
RTX PRO 5000 BLACKWELL LOCAL-SEARCH KERNEL
MICROBENCHMARKS

Optimization	Speedup	Reference
2-opt launch: 8 vs. 4 warps	1.045 $\times$	ACS, TSP100, 64 instances, population 100, 10 ACO iterations
3-opt block: 512 threads, TSP100	1.909 $\times$	old one-warp kernel
3-opt block: 512 threads, TSP500	3.511 $\times$	old one-warp kernel
Edge LSGain matrix path	25.640 $\times$	old per-edge loop

TABLE XXXI
RESIDUAL TRANSITION MINUS MATCHED REPLACEMENT (PP)

ACO	Test	Estimate [95% CI]
AS	TSP100-U	-0.606 [-0.853, -0.413]
AS	TSP500-U	-2.558 [-3.119, -1.720]
ACS	TSP100-U	+0.311 [+0.216, +0.436]
ACS	TSP500-U	+5.430 [+3.726, +6.602]
MMAS	TSP100-U	-0.679 [-1.224, +0.291]
MMAS	TSP500-U	-1.706 [-3.790, +0.452]

- [3] J. R. Koza, *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. Cambridge, MA, USA: MIT Press, 1992.
- [4] R. Poli, W. B. Langdon, and N. F. McPhee, *A Field Guide to Genetic Programming*. Lulu Enterprises, 2008, with contributions by J. R. Koza.
- [5] B.-C. Lin, Y. Mei, and M. Zhang, "Automated design of state transition rules in ant colony optimization by genetic programming: A comprehensive investigation," *Memetic Computing*, vol. 17, p. 2, 2025.
- [6] J. Tavares and F. B. Pereira, "Designing pheromone update strategies with strongly typed genetic programming," in *Genetic Programming*, ser. Lecture Notes in Computer Science, vol. 6621. Springer, 2011, pp. 85–96.
- [7] M. Dorigo and L. M. Gambardella, "Ant colony system: A cooperative learning approach to the traveling salesman problem," *IEEE Transactions on Evolutionary Computation*, vol. 1, no. 1, pp. 53–66, 1997.
- [8] T. Stützle and H. H. Hoos, "MAX–MIN ant system," *Future Generation Computer Systems*, vol. 16, no. 8, pp. 889–914, 2000.
- [9] T. Stützle, "ACOTSP, version 1.0," Software package, 2004, version 1.03 used as the algorithmic reference.
- [10] D. J. Montana, "Strongly typed genetic programming," *Evolutionary Computation*, vol. 3, no. 2, pp. 199–230, 1995.
- [11] F.-A. Fortin, F.-M. D. Rainville, M.-A. Gardner, M. Parizeau, and C. Gagné, "DEAP: Evolutionary algorithms made easy," *Journal of Machine Learning Research*, vol. 13, pp. 2171–2175, 2012.
- [12] G. Reinelt, "TSPLIB: A traveling salesman problem library," *ORSA Journal on Computing*, vol. 3, no. 4, pp. 376–384, 1991.
- [13] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.
- [14] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.
- [15] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York, NY, USA: Chapman and Hall/CRC, 1994.
- [16] C. R. Harris, K. J. Millman, S. J. van der Walt *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020.
- [17] S. K. Lam, A. Pitrou, and S. Seibert, "Numba: A LLVM-based Python JIT compiler," in *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*. ACM, 2015, pp. 1–6.
- [18] A. Paszke, S. Gross, F. Massa *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, vol. 32, 2019, pp. 8024–8035.
- [19] R. Okuta, Y. Unno, D. Nishino, S. Hido, and C. Loomis, "CuPy: A NumPy-compatible library for NVIDIA GPU calculations," in *Proceedings of the Workshop on Machine Learning Systems at NeurIPS*, 2017.

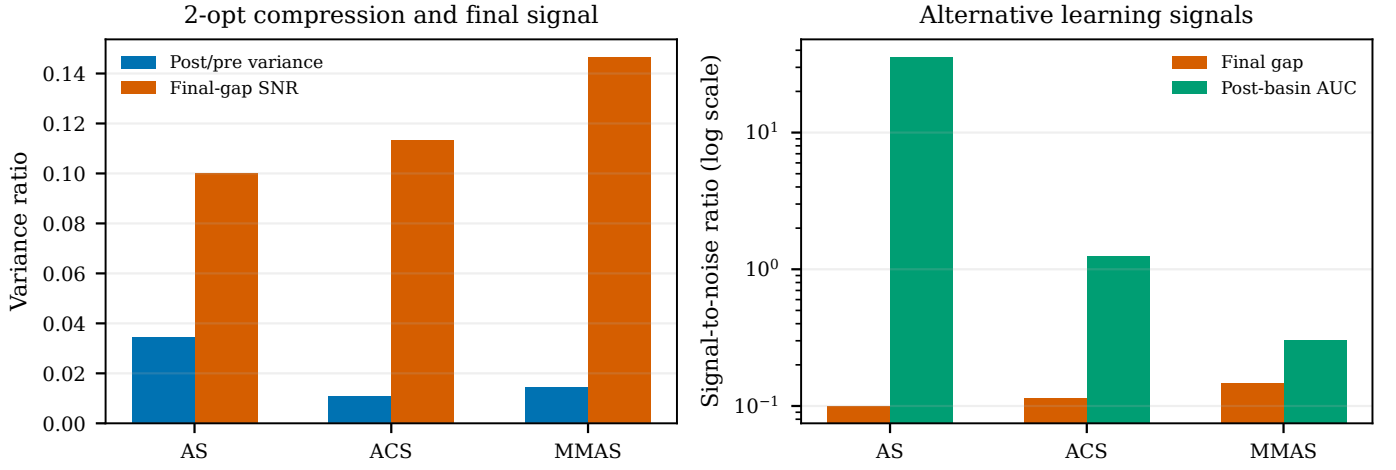


Fig. 11. Variance compression and signal-to-noise ratios after 2-opt. The right panel uses a logarithmic axis.

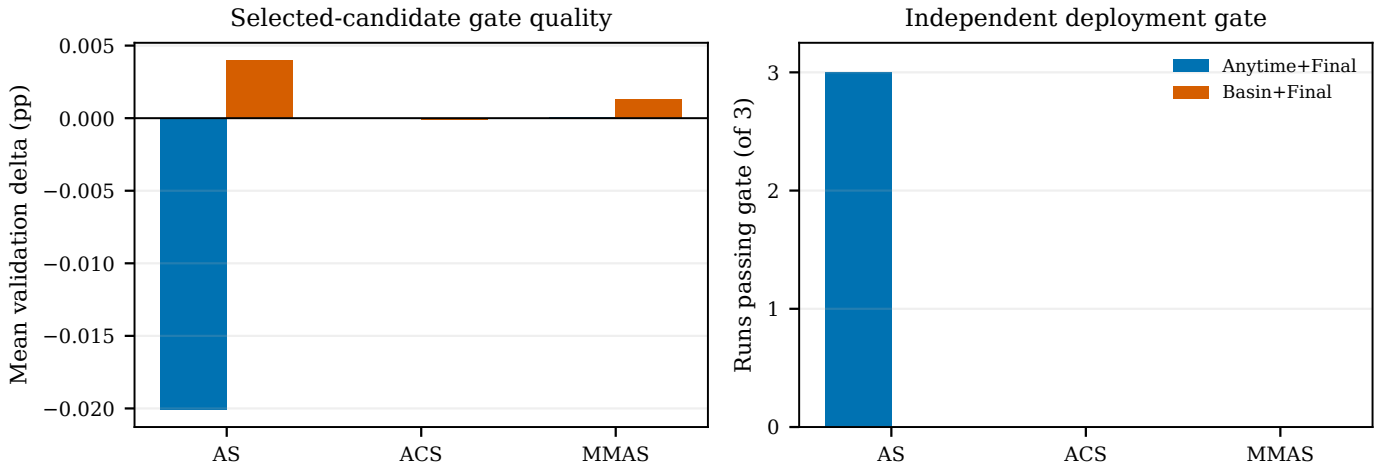


Fig. 12. Independent-gate results of the completed TSP100 fitness controls. These are validation results; no separate final-test result is inferred.

TABLE XXXII
TWO-TREE MINUS ONE-TREE CONTROLS AT 31 NODES (PP)

ACO	Test	Full-F1 – TR	Full-F1 – PH
AS	TSP100-U	-0.525 [-0.876, -0.265]	-0.068 [-0.230, +0.100]
AS	TSP500-U	-0.994 [-1.580, -0.153]	+0.795 [+0.597, +1.007]
ACS	TSP100-U	-0.006 [-0.179, +0.113]	-0.263 [-0.393, -0.167]
ACS	TSP500-U	+0.304 [-1.843, +2.336]	-2.950 [-4.575, -1.219]
MMAS	TSP100-U	-0.056 [-0.182, +0.144]	-0.828 [-0.970, -0.659]
MMAS	TSP500-U	-0.810 [-2.601, +1.187]	-1.162 [-3.075, +0.731]

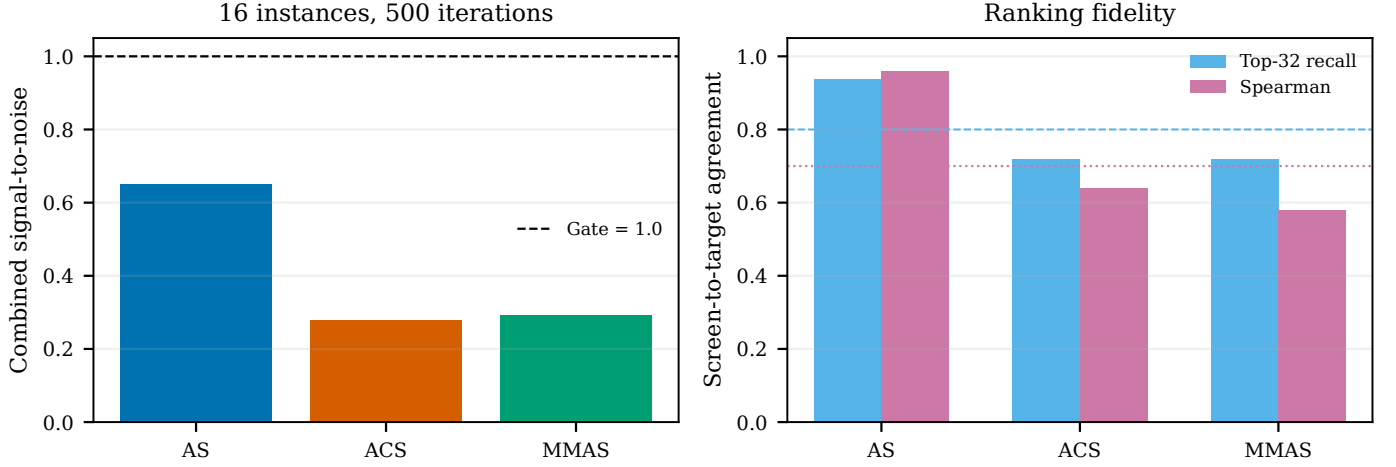


Fig. 13. Signal and ranking diagnostics for the strict TSP500 screen. No variant reaches the registered SNR threshold.

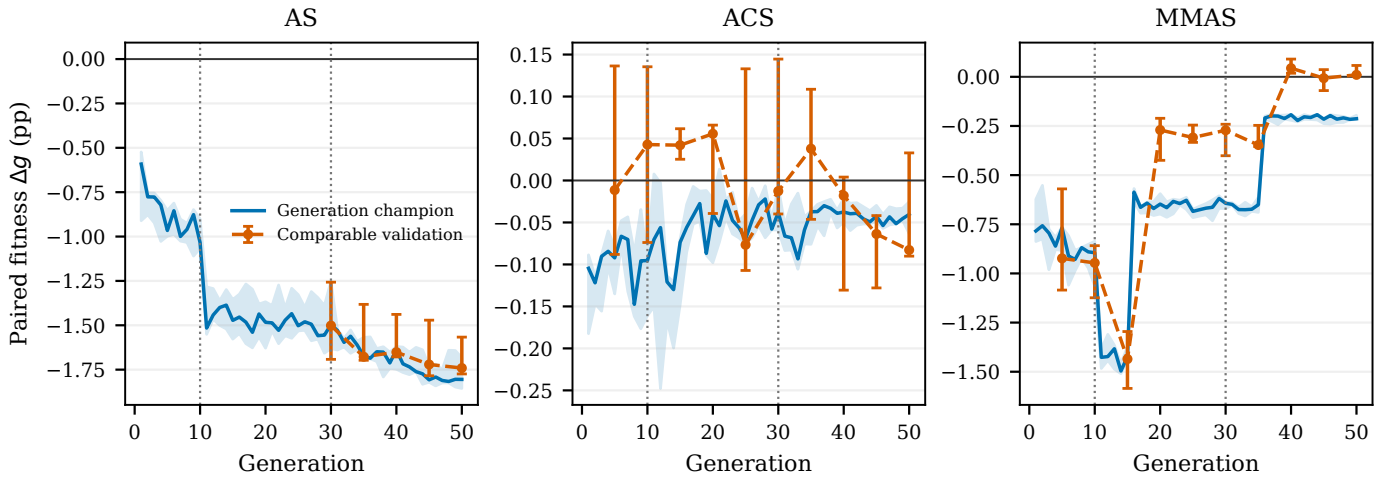


Fig. 14. TSP500 LS-aware training. Each run contributes its generation champion. Lines are medians over three GP seeds and bands are ranges. Vertical lines mark generations 10 and 30. Only validation points with the same horizon in all three runs are pooled.

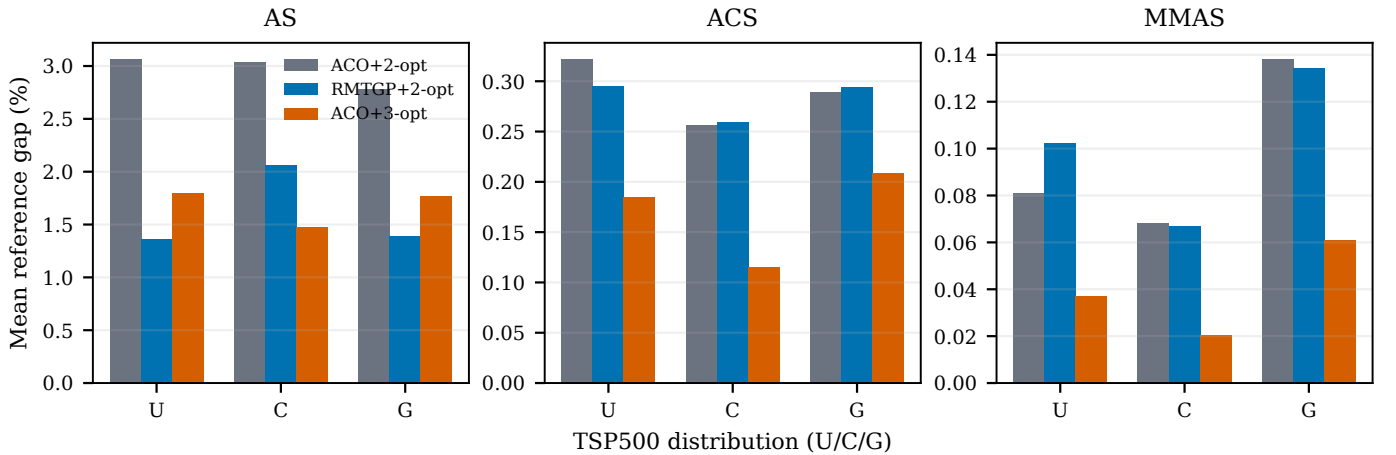


Fig. 15. Final mean gap on uniform (U), clustered (C), and Gaussian (G) TSP500.

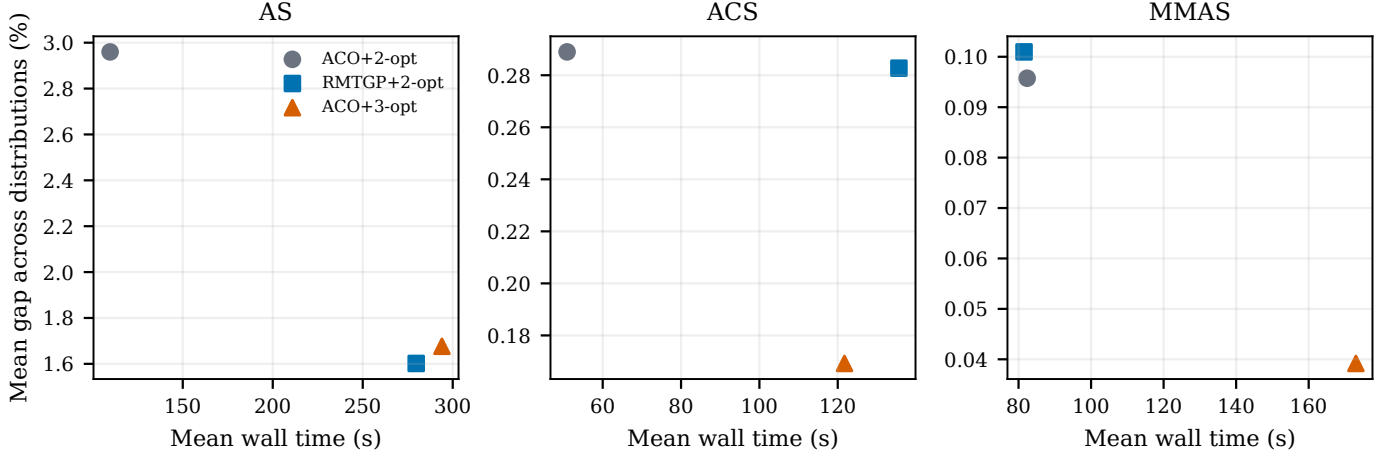


Fig. 16. Mean final-candidate runtime and mean gap across the three TSP500 distributions. Each point is one algorithmic policy averaged over distributions.

TABLE XXXIII
CORE/FULL AND F0/F1 FACTORIAL EFFECTS (PP)

ACO	Test	Full — Core	F1 — F0	Interaction
AS	TSP100-U	-0.213 [-0.534, +0.021]	+0.501 [-0.254, +0.990]	-0.410 [-0.900, +0.406]
AS	TSP500-U	-0.414 [-1.433, +0.844]	+0.951 [+0.066, +2.504]	-1.953 [-5.492, +1.437]
ACS	TSP100-U	-0.030 [-0.122, +0.071]	-0.022 [-0.058, +0.016]	-0.005 [-0.063, +0.057]
ACS	TSP500-U	+0.210 [-1.043, +2.296]	-0.219 [-1.197, +0.373]	+0.074 [-0.383, +0.548]
MMAS	TSP100-U	+0.085 [+0.017, +0.155]	+0.033 [-0.042, +0.143]	-0.004 [-0.215, +0.186]
MMAS	TSP500-U	+0.528 [+0.080, +1.163]	+0.353 [-1.106, +1.365]	+0.549 [-1.514, +2.519]

TABLE XXXIV
TREE DELETION AND CROSS-RUN PAIRING INTERVENTIONS (PP)

ACO	Test	Full — drop-TR	Full — drop-PH	Full — shuffle-1	Full — shuffle-2
AS	TSP100-U	-0.270 [-0.728, +0.000]	-1.290 [-1.576, -1.058]	-0.214 [-0.349, -0.097]	-0.292 [-0.735, +0.211]
AS	TSP500-U	-1.824 [-5.132, +0.000]	-6.090 [-7.765, -3.058]	-0.666 [-5.459, +2.957]	-0.749 [-5.407, +3.422]
ACS	TSP100-U	-0.273 [-0.321, -0.222]	-0.018 [-0.091, +0.048]	-0.032 [-0.092, +0.039]	-0.007 [-0.147, +0.121]
ACS	TSP500-U	-2.817 [-4.044, -0.854]	-0.341 [-0.659, -0.019]	-0.116 [-0.598, +0.273]	-0.236 [-0.540, +0.110]
MMAS	TSP100-U	-1.534 [-2.102, -1.006]	-0.134 [-0.255, +0.000]	+0.014 [-0.254, +0.239]	+0.026 [-0.135, +0.250]
MMAS	TSP500-U	-4.403 [-4.679, -4.140]	-1.551 [-2.834, +0.000]	-0.027 [-1.769, +2.845]	-0.029 [-2.834, +1.633]

TABLE XXXV
CAPACITY AND ARCHITECTURE CONTRASTS AT B31 AND B62

ACO	Test	B62—B31	B31 Full—TR	B31 Full—PH	B62 Full—TR	B62 Full—PH
AS	TSP100-U	-0.055 [-0.290, +0.183]	-0.525 [-0.873, -0.260]	-0.068 [-0.228, +0.102]	-0.635 [-0.756, -0.470]	-0.143 [-0.421, +0.050]
AS	TSP500-U	+0.148 [-0.627, +1.062]	-0.994 [-1.580, -0.142]	+0.795 [+0.593, +1.003]	-1.160 [-2.826, -0.173]	+0.835 [-0.195, +2.523]
ACS	TSP100-U	-0.105 [-0.194, -0.023]	-0.006 [-0.185, +0.112]	-0.263 [-0.398, -0.167]	-0.114 [-0.219, -0.022]	-0.373 [-0.420, -0.323]
ACS	TSP500-U	-0.163 [-0.875, +0.400]	+0.304 [-1.819, +2.315]	-2.950 [-4.561, -1.241]	+0.073 [-1.707, +1.393]	-3.281 [-4.189, -2.165]
MMAS	TSP100-U	-0.040 [-0.292, +0.147]	-0.056 [-0.184, +0.142]	-0.828 [-0.971, -0.663]	-0.088 [-0.149, -0.025]	-0.868 [-0.945, -0.802]
MMAS	TSP500-U	-0.271 [-2.665, +1.153]	-0.810 [-2.582, +1.200]	-1.162 [-3.061, +0.730]	-1.138 [-2.059, +0.123]	-1.433 [-2.375, +0.000]

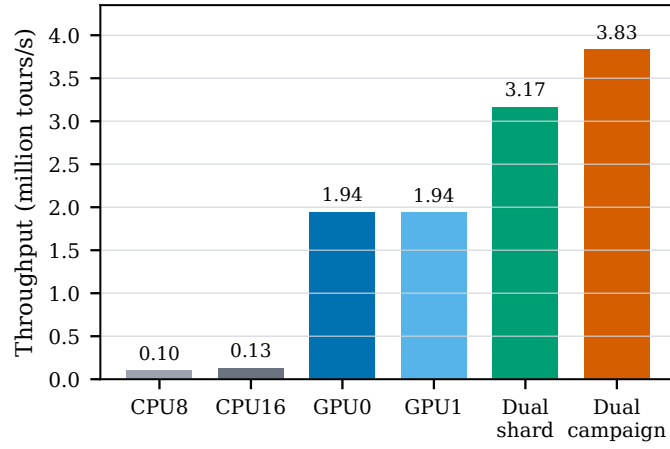


Fig. 17. Registered RTX 4000 Ada population-training throughput relative to the 16-thread CPU implementation.

TABLE XXXVI
MEDIAN SELECTED NODES AT B31/B62

ACO	TR-RGP	PH-RGP	Full-F1
AS	17/15	19/16	27/31
ACS	12/15	5/6	17/20
MMAS	19/19	7/7	19/20

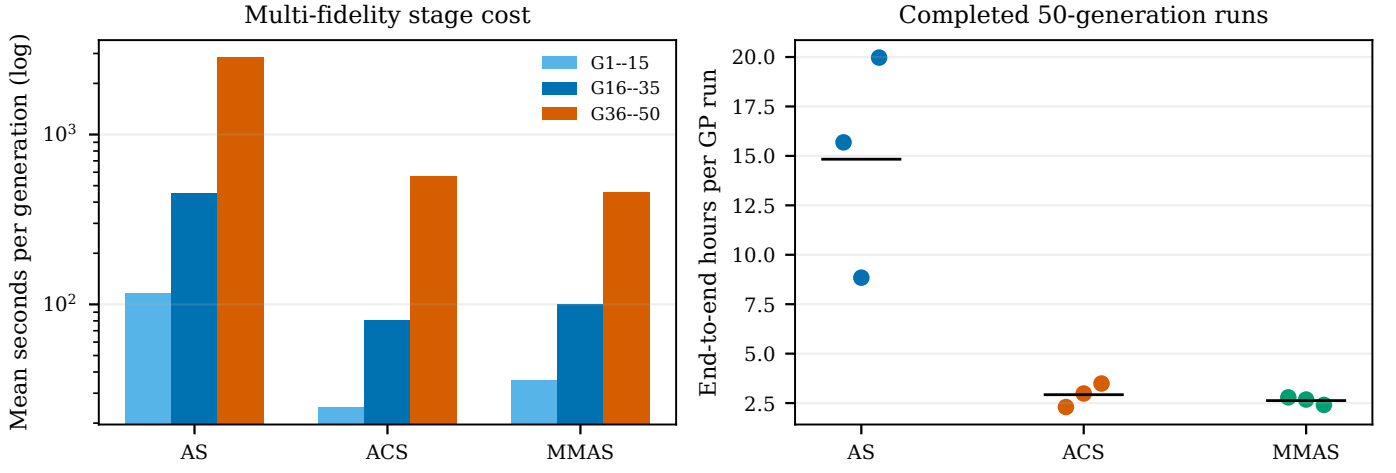


Fig. 18. Measured stage and end-to-end time for the nine completed TSP500 LS-aware GP runs.

TABLE XXXVII

COMPLETE INITIAL LOCAL-SEARCH RESULTS. VALUES ARE MEAN GAP (%); FINAL SELECTED CANDIDATES ONLY.

ACO	Test	ACO+2opt	RMTGP(no-LS)+2opt	RMTGP(LS)+2opt	ACO+3opt
AS	TSP100-U	0.0300	0.0311	0.0312	0.0043
AS	Uniform	3.0117	2.6182	2.6609	1.8274
ACS	TSP100-U	0.0154	0.0072	0.0028	0.0037
ACS	Uniform	0.3114	0.2580	0.2588	0.1989
MMAS	TSP100-U	0.0015	0.0007	0.0010	0.0007
MMAS	Uniform	0.0867	0.0939	0.0924	0.0304

TABLE XXXVIII

TSP500 FINAL-CANDIDATE WALL TIME IN SECONDS. THE LAST COLUMN IS ACO+3OPT TIME DIVIDED BY RMTGP+2OPT TIME.

ACO	Distribution	ACO+2opt	RMTGP+2opt	ACO+3opt	Ratio
AS	Uniform	108.4	264.3	264.3	1.00
AS	Cluster	113.3	288.9	344.4	1.19
AS	Gaussian	107.3	285.9	273.4	0.96
ACS	Uniform	36.7	72.6	83.6	1.15
ACS	Cluster	69.0	204.5	171.4	0.84
ACS	Gaussian	47.0	129.7	110.1	0.85
MMAS	Uniform	76.8	73.4	144.9	1.97
MMAS	Cluster	91.9	91.8	210.6	2.29
MMAS	Gaussian	78.5	79.4	163.6	2.06

TABLE XXXIX
SELECTED TSP500 LS-AWARE EXPRESSIONS. THE HASH IDENTIFIES THE SERIALIZED CANDIDATE.

Run	Hash	Transition and pheromone expressions
AS-81001	cc6491b4e380	TR: SUB(BaseConf, MAX(PDIV(POS_ONE, SUB(REta, BaseConf)), MAX(NEG(POS_ONE), PDIV(SUB(REta, BaseConf), BaseConf)))) PH: PDIV(ADD(ADD(ColonyFreq, ADD(ColonyFreq, MIN(MIN(MIN(ColonyFreq, NNRank), NNRank), POS_ONE))), MAX(MIN(ColonyFreq, MIN(ColonyFreq, NNRank)), ADD(MUL(EdgeEta, EdgeEta), MIN(MIN(ColonyFreq, MIN(ColonyFreq, NNRank)), POS_HALF))))), MUL(EdgeEta, EdgeEta))
AS-81002	a2342c879220	TR: SUB(BaseConf, ADD(ADD(SUB(NEG(BaseConf), Entropy), SUB(NEG(BaseConf), Entropy)), SUB(ADD(ADD(SUB(NEG(BaseConf), Entropy), SUB(NEG(BaseConf), Entropy)), SUB(NEG(BaseConf), Stagnation)), Stagnation))) PH: PDIV(ADD(ADD(MAX(Origin, NEG(MAX(LSGain, Origin))), ADD(ADD(MAX(NEG_HALF, Origin), ADD(Stagnation, Origin)), Origin)), ADD(PostFreq, Origin)), MUL(PostFreq, Origin))
AS-81003	f7e9fec87dd1	TR: ADD(ADD(NEG(DistRank), ADD(ADD(BaseConf, NEG(DistRank)), ADD(NEG_ONE, ADD(BaseConf, ADD(BaseConf, BaseConf))))), ADD(BaseConf, NEG(DistRank))) PH: NEG(SUB(SUB(SUB(MUL(MUL(PostFreq, POS_ONE), PostFreq), MUL(PostFreq, POS_ONE)), MUL(PDIV(POS_ONE, ADD(PostFreq, MUL(-0.61962948569942333, PostFreq))), ADD(ABS(ABS(POS_ONE)), ADD(PostFreq, MUL(-0.61962948569942333, PostFreq))))), ABS(POS_ONE)))
ACS-81001	be490a262d70	TR: NEG(SUB(DistRank, Entropy)) PH: ADD(ADD(PDIV(EdgeEta, SourceQuality), ADD(EdgeTau, EdgeEta)), MUL(PDIV(LSGain, NNRank), MIN(MAX(SourceQuality, Stagnation), ABS(EdgeEta))))
ACS-81002	e7b1953ac8d7	TR: ADD(RTau, PDIV(PDIV(BaseConf, BaseConf), BaseConf)) PH: MUL(MAX(NEG(ADD(EdgeEta, SUB(Prefreq, LSGain))), MAX(SUB(Prefreq, EdgeEta), SUB(MAX(SUB(Prefreq, EdgeEta), SUB(Origin, EdgeEta)), EdgeEta))), ADD(ADD(EdgeEta, EdgeEta), ADD(EdgeEta, Prefreq)))
ACS-81003	df276a0af5e6	TR: ZERO_TR PH: MUL(SUB(ABS(PDIV(ABS(POS_ONE), EdgeEta)), MUL(NEG_HALF, MAX(ADD(Stagnation, EdgeTau), NEG(LSGain)))), MUL(PDIV(NNRank, POS_ONE), ADD(PDIV(NNRank, EdgeEta), EdgeEta)))
MMAS-81001	7aa978afdc7b	TR: ZERO_TR PH: ADD(PDIV(SourceQuality, PostFreq), MUL(ADD(ColonyFreq, PostFreq), SourceQuality))
MMAS-81002	0d8baf1ce7b8	TR: ADD(MAX(ZERO_TR, ADD(ADD(ADD(BaseConf, BaseConf), BaseConf), ADD(Stagnation, BaseConf))), MIN(ADD(ADD(ZERO_TR, BaseConf), MIN(ZERO_TR, Stagnation)), BaseConf)) PH: NEG(NEG(SUB(ADD(Prefreq, MAX(ADD(EdgeTau, Origin), MAX(ZERO_PH, Stagnation))), SUB(SUB(NEG(Prefreq), EdgeTau), MAX(Stagnation, NNRank)))))
MMAS-81003	bd0c4c5514c6	TR: ZERO_TR PH: MIN(PDIV(NNRank, PostFreq), SUB(POS_ONE, PDIV(NEG(MAX(TauHeadroom, ADD(ZERO_PH, Origin))), TauHeadroom)))

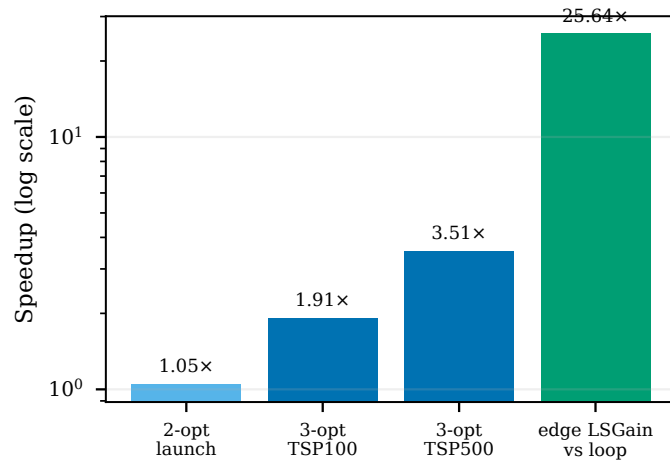


Fig. 19. Blackwell kernel speedups. The protocols differ, so the bars are not combined statistically.

TABLE XL
WARM SINGLE-PROGRAM OVERHEAD RELATIVE TO ORIGINAL ACO

ACO	TSP50	TSP100	TSP500	TSP1000
AS	+467.11%	+340.88%	+136.02%	+122.09%
ACS	+53.83%	+63.27%	+112.91%	+123.20%
MMAS	+487.25%	+418.58%	+264.93%	+262.94%

TABLE XLI
SEMANTIC-TERMINAL MICROBENCHMARK ON RTX PRO 5000
BLACKWELL

Path	Time (s)	Relative factor
EdgeEta	1.602	1.000
PreFreq	1.606	0.998
PostFreq	1.613	0.993
LSGain	2.217	25.640

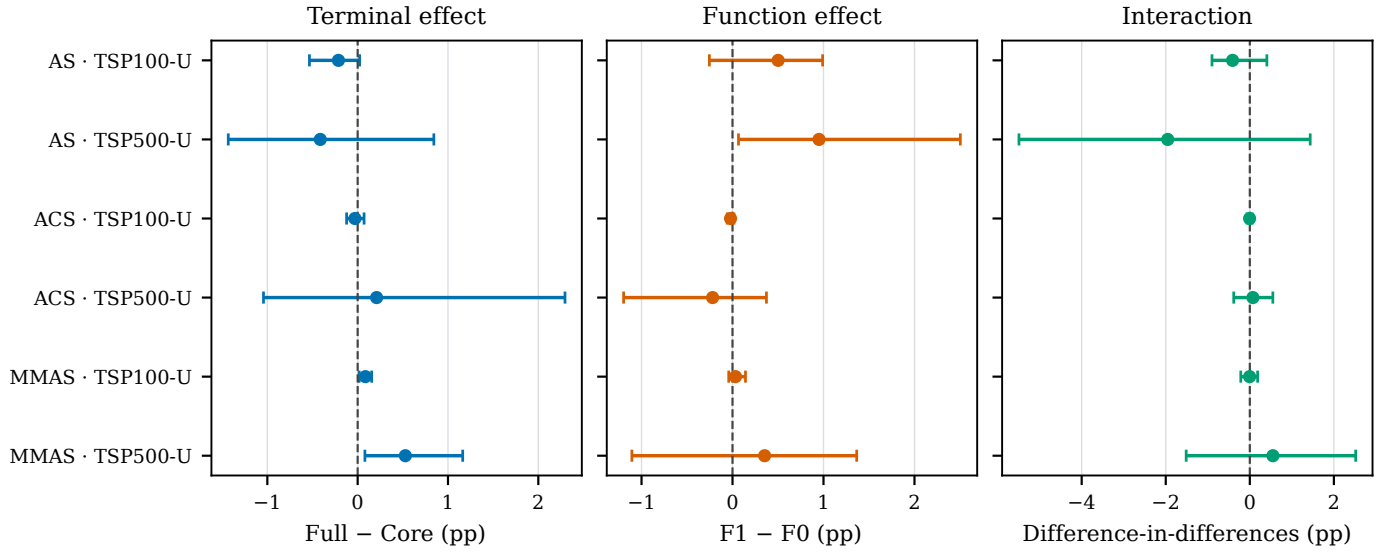


Fig. 20. Full factorial contrasts. No interaction interval excludes zero.

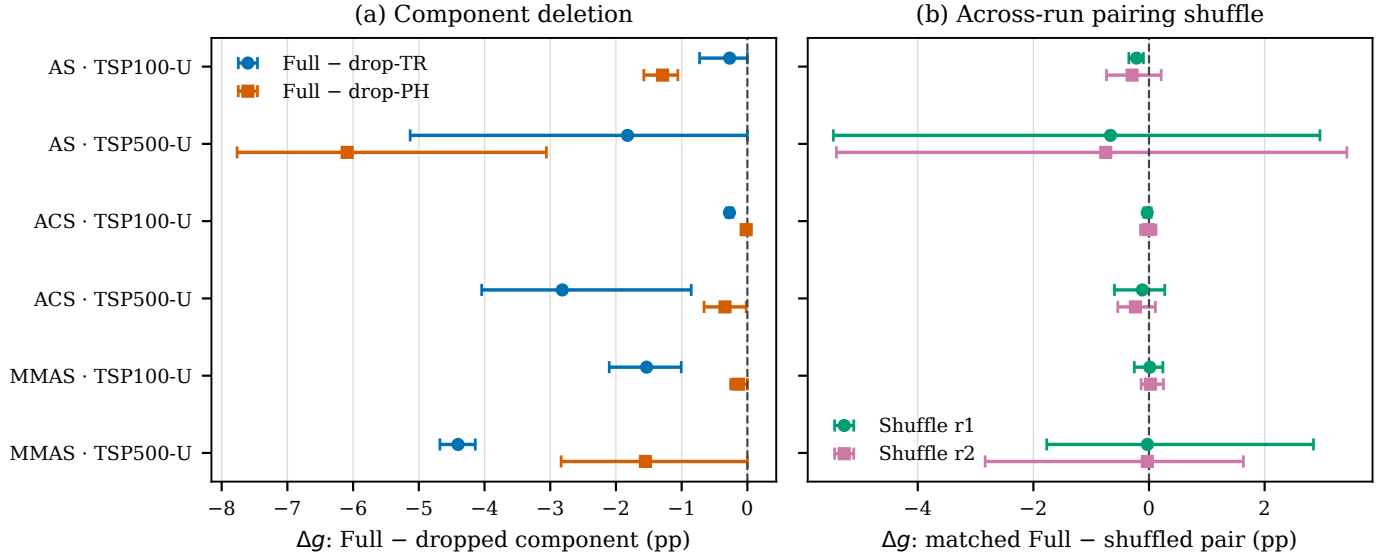


Fig. 21. Full post-hoc deletion and shuffle contrasts.

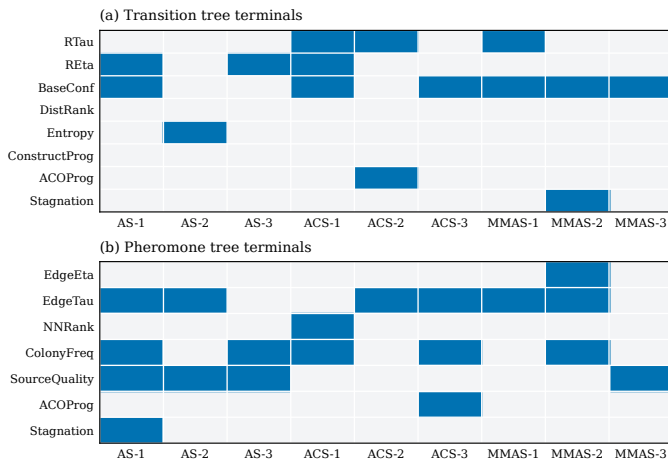


Fig. 22. Syntactic terminal use in the nine no-LS Full-F1 champions.

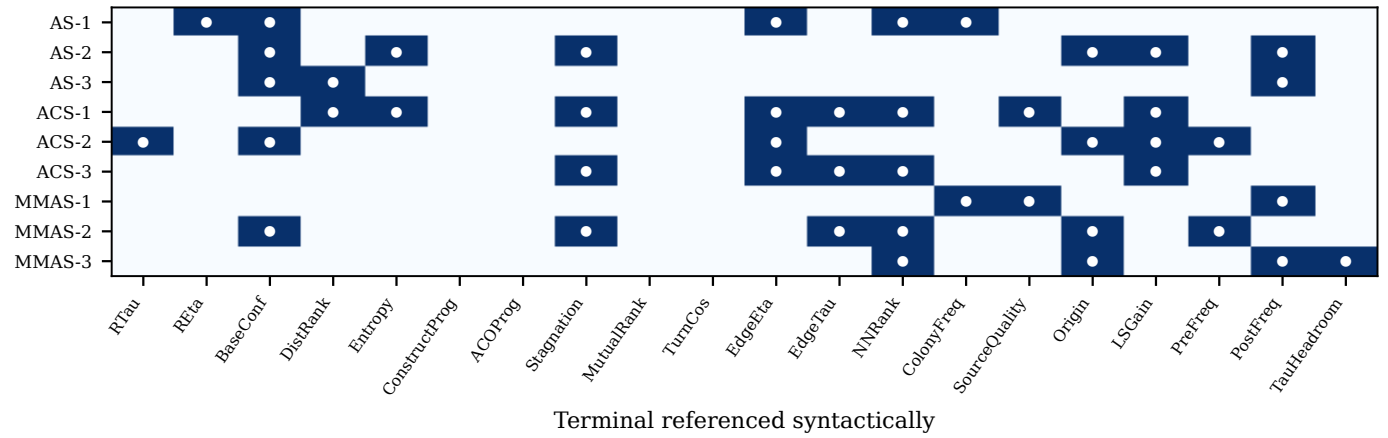


Fig. 23. Syntactic terminal use in the nine selected TSP500 LS-aware candidates. Occurrence is not a causal effect.